



MULTI-DISEASE PREDICTION SYSTEM

INTERNSHIP REPORT

Submitted By

RAGHUL RAJA M - (512223104075)

In partial fulfilment for the award of

BACHELOR OF ENGINEERING

(COMPUTER SCIENCE AND ENGINEERING)

S.K.P. ENGINEERING COLLEGE

THIRUVANNAMALAI-606601



ANNA UNIVERSITY, CHENNAI 600025

JULY – AUGUST 2026



TVK Technologies

Internship Completion Letter

Date: 08/08/2026

This is to certify that **Mr. RAGHUL RAJA M (512223104075)**, a student of **SKP Engineering College** has successfully completed the internship in **AI With Data Science** at **TVK Technologies** from **08 July 2026 to 07 August 2026** under the supervision of **PRIYATHARSHAN A / Program Manager**.

During the internship period, he worked on the project titled **“Build a multi-disease prediction system using machine learning”** and demonstrated sincerity, dedication, and a strong willingness to learn. He actively contributed to project development, task execution, documentation, and team collaboration throughout the internship.

His performance was professional, punctual, and well-structured. He displayed good problem-solving abilities, adaptability, and teamwork skills, which were appreciated by the organization.

We wish him continued success in all future endeavors and look forward to supporting his career journey.

Best wishes,



CEO / FOUNDER



A. Priyatharshan.

PROGRAM MANAGER

Phone: +91 94439 38850

Website: www.tvktechs.com

Email: hr@tvktechs.com

BONAFIDE / INTERNSHIP CERTIFICATE

This page is provided as a formal certificate template for institutional verification. The authorized college / department signatory may replace the placeholder text and signature blocks with the officially issued wording and seal.

Student Name	RAGHUL RAJA M
Register Number	512223104075
Project	MULTI-DISEASE PREDICTION SYSTEM USING MACHINE LEARNING
Organization	TVK TECHNOLOGIES
Internship Period	08 JULY 2026 - 07 AUGUST 2026

Faculty Supervisor

Signature

Head of the Department

Signature

Principal

Signature / Seal

DECLARATION

I, RAGHUL RAJA M (Register Number: 512223104075), a student of B.E. Computer Science and Engineering at S.K.P. Engineering College, Thiruvannamalai, declare that the internship report entitled “MULTI-DISEASE PREDICTION SYSTEM USING MACHINE LEARNING” documents the academic work undertaken during my one-month internship at TVK Technologies in the AI with Data Science domain from 08 July 2026 to 07 August 2026.

The report has been prepared for academic evaluation and presents the design, implementation approach, machine-learning workflow, testing considerations, limitations, and outcomes associated with the internship project. Technical implementation details are described only to the extent supported by the project information supplied for this report.

The predictions produced by the application are not medical diagnoses and the system is not intended to replace doctors, hospitals, laboratory testing, or professional medical advice.

RAGHUL RAJA M

Date: 24 AUGUST 2026

ACKNOWLEDGEMENT

I express my sincere gratitude to S.K.P. Engineering College, Thiruvannamalai, and the Department of Computer Science and Engineering for providing the academic environment and guidance required to undertake this internship project. I am thankful to TVK Technologies for providing the opportunity to work in the AI with Data Science domain and to gain practical exposure to machine-learning application development.

I extend my special thanks to PRIYATHARSHAN A, Program Manager and internship supervisor, for the direction, encouragement, review, and professional guidance provided throughout the internship period. The practical experience of working through dataset preparation, model development, model evaluation, application development, testing, and documentation contributed significantly to my learning.

I also thank the faculty members, mentors, classmates, friends, and family members whose encouragement and support helped me complete the internship work and prepare this report. Their feedback and motivation were valuable throughout the project.

Finally, I acknowledge the open-source Python and machine-learning ecosystem and the public datasets and documentation resources that support academic experimentation in data science.

ABSTRACT

Healthcare-related datasets contain structured information that can be used to demonstrate how machine-learning classification techniques support educational risk-prediction workflows. During the AI with Data Science internship at TVK Technologies, a multi-disease prediction system was developed as an educational web application using Python and Streamlit. The project brings together five disease-specific prediction modules covering Heart Disease, Diabetes, Parkinson's Disease, Liver Disease, and Kidney Disease. Each module accepts a defined set of health or biomedical parameters and uses a disease-specific trained machine-learning model to generate a prediction result and associated probability/risk interpretation.

The project follows an end-to-end machine-learning workflow beginning with dataset loading and validation, followed by data cleaning, encoding, feature preparation, scaling where required, train/test processing, candidate-model training, hyperparameter tuning, cross-validation, performance evaluation, and model selection. The training workflow compares Random Forest, Logistic Regression, Support Vector Machine, Decision Tree, Gradient Boosting, and Extra Trees so that the best-performing algorithm can be selected separately for each disease rather than assuming one model is universally optimal. The trained models and preprocessing artifacts are serialized and reused by the Streamlit application during prediction.

Beyond prediction, the application provides a practical analytics layer. It includes disease-wise prediction counts, detection versus non-detection summaries, risk-level distributions, probability visualizations, prediction timelines, model metric visualization, radar charts, feature-importance views, prediction history, dataset inspection, system information, settings, and report export in CSV, Excel, and PDF formats. The interface is organized into Prediction, Analytics, and Information areas and supports a dark/light theme preference through a persistent `user_settings.json` configuration file.

The internship project demonstrates how machine-learning models, data-processing libraries, interactive visualization, and a lightweight web framework can be combined into a single educational application. The result is intended for academic learning, experimentation, and research-oriented demonstration. It is not a clinical decision-support system, and its predictions must not be interpreted as medical diagnoses. Dataset quality, feature coverage, training methodology, population differences, and the absence of clinical validation limit the generalizability of the results.

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT LIST OF FIGURES LIST OF ABBREVIATIONS	
1.	CHAPTER 1 : INTRODUCTION 1.1 INTRODUCTION 1.2 BACKGROUND 1.3 PROBLEM STATEMENT 1.4 PURPOSE 1.5 OBJECTIVES 1.6 PROJECT OVERVIEW 1.7 SCOPE OF THE PROJECT 1.8 SIGNIFICANCE OF THE PROJECT 1.9 TOOLS AND TECHNOLOGIES	9

2.	CHAPTER 2 : FEASIBILITY 2.1 INTRODUCTION 2.2 OPERATIONLA FEASIBILITY 2.3 TECHNICAL FEASIBILITY 2.4 ECONOMIC FEASIBILITY 2.5 SCHEDULE FEASIBILITY 2.6 EXISITING SYSTEM 2.7 LIMITATION OF EXISTING SYSTEM 2.8 PROPOSED SYSTEM 2.9 ADVANTAGES 2.10 LITERATURE / RELATED WORK	12
----	--	----

3.	CHAPTER 3 :STUDY OF THE SYSTEM 3.1 SYSTEM OVERVIEW 3.2 SYSTEM USERS 3.3 SYSTEM MODULES 3.4 DISEASE PREDICTION MODULES 3.5 ANALYTICS MODULE 3.6 MODEL COMPARISON MODULE 3.7 PERFORMANCE DASHBOARD 3.8 FEATURE IMPORTANCE MODULE 3.9 PREDICTION HISTROY 3.10 REPORT EXPORT 3.11 DATASET INFORMATION 3.12 SYSTEM INFORMATION 3.13 SETTING MODULES	14

4.	CHAPTER 4: SOFTWARE REQUIREMENT STUDY 4.1 PRODUCT DESCRIPTION 4.2 USER CHARACTERISTICS 4.3 HARDWARE AND SOFTWARE REQUIREMENTS 4.4 FUNCTIONAL REQUIREMENTS 4.5 NON-FUNCTIONAL REQUIREMENTS 4.6 PERFORMANCE REQUIRMENTS 4.7 USABILITY REQUIREMENTS 4.8 SECURITY CONSIDERATION 4.9 RELIABILITY & MAINTAINBILITY 4.10 PORTABILITY	17
5.	CHAPTER 5: SYSTEM DESIGN 5.1 REQUIREMENT ANALYSIS 5.2 SYSTEM ANALYSIS 5.3 SYSTEM ARCHITECTURE 5.4 OVERALL SYSTEM WORKFLOW 5.5 MACHINE LEARNING PIPLINE 5.6 APPILICATION ARCHITECTURE 5.7 UML DIAGRAM 5.8 USE CASE & ACTIVITY DIAGRAM 5.9 SEQUENCE & COMPONENT DIAGRAM 5.10 DATA FLOW DIAGRAM	20

6.	CHAPTER 6: DATABASE DESIGN 6.1 DATABASE DESIGN 6.2 DATASET ARCHITECTURE 6.3 DATASET DESCRIPTION 6.4 HEART & DIABETES DATASETS 6.5 PARKINSONS & LIVER & KIDNEY DATASETS 6.6 FEATURE REPRESENTATION 6.7 TARGET VARIABLES 6.8 DATA PREPROCESSING 6.9 ENCODING & SCALING &FEATURE 6.10 MODEL ARTIFICT & PREDICTION STORAGE	28
7.	CHAPTER 7: LIMITATION 7.1 PROJECT LIMITATIONS 7.2 DATASET LIMITATIONSS 7.3 MODEL LIMITATIONS 7.4 PREDICTION LIMITATIONS 7.5 GENRALIZATION LIMITATIONS 7.6 MEDICAL INTERPRETATION 7.7 FUTURE IMPROVEMENT	32
8.	CHAPTER 8: IMPLEMENTATIONS 8.1 IMPLEMENTATIONS OVERBIEW 8.2 ENVIRONMENT SETUP 8.3 DATASET PREPRATION 8.4 MODEL TRAINING	34

	8.5 MODEL SERIALIZATION 8.6 STREAMLIT APPLICATION DEV 8.7 PRODUCTION MODULE 8.8 ANALYTICS & HISTORY 8.9 MODEL COMPARISON 8.10 REPORT,EXPORT,SETTINGS 8.11 TESTING & DEPLOYEMENT	
9.	CHAPTER 9: SOURCE CODE 9.1 APPLICATION ENTRY POINT 9.2 DATASET LOADING 9.3 DATA PREPOCESSING 9.4MODEL LOADING 9.5 PREDICTION FUNCTION 9.6 INPUT VALIDATION 9.7 RISK INTERPRETATION 9.8 ANALYTICS 9.9 MODEL COMPARISONS 9.10 PREDICTION HISTROY 9.11 REPORT EXPORT 9.12 TESTING	37
10.	CHAPTER 10: CONCLUSION 10.1 PROJECT & LEARNING OUTCOMES 10.2 FUTURE ENHANCEMENTS 10.3 FINAL REMARKS	64

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
ML	Machine Learning
UI	User Interface
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
PDF	Portable Document Format
ROC	Receiver Operating Characteristic
AUC	Area Under the Curve
F1	F1 Score
SVM	Support Vector Machine
RF	Random Forest
ECG	Electrocardiogram
BMI	Body Mass Index
HbA1c	Glycated Hemoglobin
CKD	Chronic Kidney Disease

CHAPTER 1 - INTRODUCTION

1.1 Introduction

The increasing availability of structured healthcare datasets has created opportunities to apply machine-learning techniques in educational and research applications. Machine-learning classification models can identify patterns from previously labelled data and use these patterns to predict the class of new input records.

This internship project focuses on developing a **Multi-Disease Prediction System** that integrates five disease prediction modules: **Heart Disease, Diabetes, Parkinson's Disease, Liver Disease, and Kidney Disease**. The system is developed using **Python and Streamlit**, providing an interactive interface for entering patient-related features and generating prediction results.

The project demonstrates the basic machine-learning workflow, including **data preprocessing, model training, model evaluation, and prediction**. It also provides supporting features such as visualization and prediction history to improve the understanding of the results.

The main objective of this project is to provide a practical and educational demonstration of how machine-learning techniques can be applied to healthcare datasets. The system is developed for **learning and research purposes only** and is not intended to replace professional medical diagnosis.

1.2 Background

Traditional health-risk assessment in academic demonstrations often requires separate scripts, separate datasets, or disconnected notebooks. A unified application makes it easier to demonstrate the relationship between data preprocessing, model training, model selection, prediction, visualization, and reporting. The project therefore combines the machine-learning pipeline with a user-facing interface and an analytics layer.

1.3 Problem Statement

The project addresses the need for an educational application that can demonstrate multi-disease machine-learning prediction using structured health and biomedical features, while also providing model evaluation, analytics, history, and export facilities in a single interface. The challenge is not only to train classifiers, but to ensure that the same preprocessing assumptions used during training are consistently applied during prediction.

1.4 Purpose

The purpose is to build a practical, maintainable, and demonstrable machine-learning web application that helps a learner understand an end-to-end prediction workflow across five different datasets and feature spaces.

1.5 Objectives

- Develop disease-specific classification workflows for Heart Disease, Diabetes, Parkinson's Disease, Liver Disease, and Kidney Disease.
- Prepare and validate the input data and preprocessing artifacts required by each model.
- Compare multiple machine-learning algorithms and select the best-performing model separately for each disease.
- Provide an interactive Streamlit interface for user input and result visualization.
- Provide analytics, performance dashboards, feature-importance views, prediction history, and export features.
- Document limitations and clearly position the application as an educational/research system, not as a medical diagnosis tool.

1.6 Project Overview

The system consists of a training workflow and an interactive inference workflow. The training workflow is coordinated by `train_models.py` and creates disease-specific serialized models and preprocessing artifacts. The application workflow is managed through `app.py` and exposes prediction modules, dashboards, analytics, information pages, settings, and export functions.

Live application: <https://organsenseai.streamlit.app/>

GitHub repository: <https://github.com/raghul692/Multi-Prediction-Sytem>

1.7 Scope of the Project

The scope covers dataset-driven binary classification demonstrations, preprocessing, model comparison, model serialization, web-based prediction, analytics, history, and export. It does not include clinical diagnosis, hospital integration, electronic medical records, laboratory device integration, or clinical validation.

1.8 Significance of the Project

The project is significant as an internship learning exercise because it combines data science and software development in one artifact. It provides a repeatable pattern for loading datasets,

preparing features, comparing classifiers, storing trained models, and exposing them through a simple web interface.

1.9 Tools and Technologies

Category	Technology / Version	Role
Language	Python 3.10+	Application and model-development language
Web framework	Streamlit 1.39	Interactive web UI and application runtime
Machine learning	Scikit-learn 1.5	Model training, tuning, validation and evaluation
Data processing	Pandas, NumPy	Tabular data handling and numerical computation
Visualization	Matplotlib, Seaborn, Plotly	Charts, analytics and metric visualization
Export	fpdf2, Pandas	PDF, CSV and Excel export
Serialization	Pickle	Storage and reuse of trained models/artifacts
Testing	pytest	Basic test automation and validation

CHAPTER 2 - FEASIBILITY STUDY

2.1 Introduction

Feasibility study evaluates whether the project can be developed and used within the available internship scope. Because the application is an educational machine-learning web application, feasibility depends mainly on software availability, dataset accessibility, computational requirements, development time, and maintainability.

2.2 Operational Feasibility

Operationally, the application is simple to use because the main interaction is form-based. A user selects a disease, enters the required parameters, submits the data, and reviews the generated result. The dashboard and analytics pages provide additional interpretation of historical application activity.

2.3 Technical Feasibility

The project is technically feasible using commonly available Python libraries. Streamlit provides the user interface, while Scikit-learn supports the classification models and evaluation workflow. Pandas and NumPy support data preparation, while plotting libraries support visualization. The serialized artifact approach also avoids retraining a model on every prediction request.

2.4 Economic Feasibility

The project has low direct software cost because it is built primarily with open-source Python libraries and educational datasets. The main resource requirements are development time, a computer capable of running Python and the required libraries, and an internet connection for deployment and repository access where applicable.

2.5 Schedule Feasibility

The internship duration was one month, from 08 July 2026 to 07 August 2026. Within this schedule, the work was organized around understanding the problem, preparing datasets, training and comparing models, building the interface, testing the model artifacts, adding analytics and export features, and completing documentation.

2.6 Existing System

A common academic approach is to run separate notebooks or scripts for each disease dataset. Such workflows are useful for experimentation but may not provide a consistent user interface, common history, shared analytics, or export facilities.

2.7 Limitations of Existing System

- Fragmented user experience across separate scripts or notebooks.
- Limited visibility into model comparison and application-level behavior.
- Difficult to maintain a common prediction history and reporting workflow.
- Little opportunity to demonstrate interactive analytics in the same interface.

2.8 Proposed System

The proposed system unifies five disease-specific prediction modules within a Streamlit application. It combines reusable preprocessing artifacts, serialized models, model-comparison reporting, dashboards, feature importance, prediction history, dataset inspection, system information, settings, and export capabilities.

2.9 Advantages of Proposed System

- Single application for five disease prediction modules.
- Disease-specific model selection rather than one universal algorithm assumption.
- Consistent preprocessing and artifact reuse during inference.
- Interactive analytics and visualization.
- Searchable prediction history and export functionality.
- Suitable architecture for academic demonstrations and future experimentation.

2.10 Literature / Related Work

Machine-learning classification is widely used for supervised predictive modelling, where algorithms learn a mapping from features to labelled classes. Scikit-learn provides standard implementations and evaluation utilities for algorithms such as logistic regression, random forests, support vector machines, decision trees, gradient boosting, and extra trees. The project follows this standard supervised-learning pattern while emphasizing application integration.

CHAPTER 3 - STUDY OF THE SYSTEM

3.1 System Overview

The system is composed of three high-level areas: Prediction, Analytics, and Information. Prediction modules accept disease-specific inputs and invoke the corresponding trained model. Analytics pages summarize accumulated prediction activity and model performance. Information pages expose dataset, environment, configuration, and application details.

3.2 System Users

The principal user is an academic learner, project reviewer, or demonstrator who wants to explore machine-learning prediction workflows. The system is not designed as a clinical product for physicians or patients, and the application should not be represented as one.

3.3 System Modules

Area	Modules	Purpose
Prediction	Home, Heart Disease, Diabetes, Parkinson's, Liver Disease, Kidney Disease	Collect inputs and execute disease-specific predictions
Analytics	Dashboard, Analytics, Model Comparison, Performance Dashboard, Feature Importance, Prediction History	Explore prediction activity, performance and model behavior
Information	Dataset Information, System Information, Settings, About	Explain data, environment, preferences and application purpose

3.4 Disease Prediction Module

The prediction module is disease-specific because each dataset has a different feature set and representation. A selected model is loaded with the preprocessing artifacts required for its input format. The application validates inputs, prepares the feature vector, performs inference, and presents a prediction probability and risk interpretation where supported by the project design.

Disease	Representative Features	Target
Heart Disease	Age, Sex, Chest Pain Type, Resting Blood Pressure, Cholesterol, Fasting Blood Sugar, Resting ECG, Maximum	HeartDisease

Disease	Representative Features	Target
	Heart Rate, Exercise-Induced Angina, Oldpeak, and ST Slope	
Diabetes	Gender, Age, Hypertension, Heart Disease, Smoking History, BMI, HbA1c Level, Blood Glucose Level	diabetes
Parkinson's Disease	22 biomedical/voice measurements used by the supplied Parkinson's dataset	status
Liver Disease	Age, Gender, bilirubin values, liver enzymes, protein-related and other biochemical parameters	Dataset
Kidney Disease	Blood Pressure, Specific Gravity, Albumin, Sugar, Blood Glucose, Blood Urea, Serum Creatinine, Sodium, Potassium, Hemoglobin and other kidney-related parameters	classification

3.5 Analytics Module

The analytics layer converts application history and evaluation information into interpretable charts. Documented views include disease distribution, detection versus non-detection, risk-level distribution, summary cards, probability distributions, prediction timelines, interactive charts, model metric visualizations, and radar charts.

3.6 Model Comparison Module

Model comparison is implemented as a per-disease selection problem. Candidate algorithms include Random Forest, Logistic Regression, Support Vector Machine, Decision Tree, Gradient Boosting, and Extra Trees. The system is designed to select the best-performing candidate for each disease after evaluation instead of assuming the same algorithm will be best across all datasets.

3.7 Performance Dashboard

The performance dashboard documents Accuracy, Precision, Recall, F1 Score, ROC-AUC, and Confusion Matrix results. Exact numerical values are intentionally not fabricated in this report because the supplied project information does not include verified final metric values. The

generated evaluation report and in-application dashboard should be treated as the source for the actual values.

3.8 Feature Importance Module

Feature importance or contribution views help identify which input variables are more influential to a trained model according to the available model-specific interpretation method. Such visualizations describe model behavior and should not be interpreted as clinical causation.

3.9 Prediction History

Prediction history is automatically saved as application records. The project supports searching and filtering historical entries and exporting the resulting records. This provides a practical way to inspect application use over time without retraining the models.

3.10 Report Export

The application supports CSV, Excel, and PDF exports. PDF generation uses `fpdf2`, while tabular CSV/Excel export is handled through Pandas-supported output workflows.

3.11 Dataset Information

Users can inspect dataset information and previews through the application. This feature is important for educational transparency because it allows the user to understand which datasets feed the prediction workflows.

3.12 System Information

The system information view exposes dependency and environment information so that the user or reviewer can understand the software context in which the application is running.

3.13 Settings Module

User preferences such as theme selection are persisted in `user_settings.json`. Persistent settings improve continuity between application sessions and demonstrate lightweight configuration management.

CHAPTER 4 - SOFTWARE REQUIREMENT STUDY

4.1 Product Description

The product is an educational machine-learning web application delivered through Streamlit. It combines offline model training and interactive online inference in a single project structure. The application depends on pre-trained model artifacts and associated preprocessing objects at runtime.

4.2 User Characteristics

- Basic ability to operate a web browser and understand form fields.
- Academic interest in machine learning or data science.
- Ability to interpret prediction results as probabilistic/educational outputs rather than medical diagnosis.

4.3 Hardware Requirements

Component	Recommended / Practical Requirement
Processor	Modern multi-core CPU suitable for Python and Scikit-learn workloads
Memory	8 GB RAM or above recommended for comfortable development
Storage	At least 2 GB free space for project, environments and model artifacts
Display	1366 x 768 or higher
Network	Internet connection for repository/deployment access; local execution can operate without continuous network use

4.4 Software Requirements

Software	Requirement
Operating System	Windows, Linux, or macOS capable of running Python 3.10+
Python	3.10+
Streamlit	1.39
Scikit-learn	1.5
Pandas / NumPy	Project runtime dependencies

Software	Requirement
Matplotlib / Seaborn / Plotly	Visualization
fpdf2	PDF generation
pytest	Testing

4.5 Functional Requirements

- The system shall allow the user to choose one of the five disease modules.
- The system shall collect and validate disease-specific input features.
- The system shall load the corresponding model and preprocessing artifacts.
- The system shall generate a prediction and display an associated probability/risk interpretation where available.
- The system shall store prediction history records.
- The system shall provide dashboard and analytics views.
- The system shall provide model comparison and performance information.
- The system shall support export to CSV, Excel, and PDF.
- The system shall allow the user to inspect datasets and system information.
- The system shall persist theme settings in user_settings.json.

4.6 Non-Functional Requirements

- Usability: simple navigation and clear grouping of modules.
- Maintainability: training, validation, application and report generation are separated into identifiable files.
- Reliability: trained models and preprocessing artifacts should be loaded consistently.
- Portability: the application should run wherever the documented Python dependencies are available.
- Transparency: model performance and limitations should be visible to the user.

4.7 Performance Requirements

Prediction requests should be responsive once the application and model artifacts are loaded. Training time is expected to be higher than inference time because it includes repeated model fitting, tuning, cross-validation, and evaluation. Exact runtime benchmarks are not asserted because they depend on hardware and dataset characteristics.

4.8 Usability Requirements

The interface should use grouped navigation, readable labels, meaningful section names, visible validation messages, and consistent result presentation. A dark/light theme option is included to support user preference.

4.9 Security Considerations

The project should avoid presenting prediction output as clinical truth. User-provided information should be handled carefully, and exported history files should be treated as potentially sensitive. The application should not claim secure clinical-data compliance because such validation is outside the project scope.

4.10 Reliability

Reliability depends on correct model loading, consistent preprocessing, valid inputs, and stable application dependencies. Verification scripts are included to check model artifacts and related behavior.

4.11 Maintainability

The project separates major responsibilities across `app.py`, `train_models.py`, `test_models.py`, `verify_models.py`, report-generation utilities, datasets, models, and configuration/history files. This organization supports later modification and debugging.

4.12 Portability

Because the solution is implemented in Python and Streamlit, it can be moved between compatible development environments as long as required library versions and serialized artifacts remain compatible.

CHAPTER 5 - SYSTEM DESIGN

5.1 Requirement Analysis

The analysis phase converts the project objectives into application modules, data requirements, model requirements, and operational workflows. Special attention is given to preserving the feature order and preprocessing assumptions expected by each disease-specific model.

5.2 System Analysis

The system consists of a training side and an inference side. The training side transforms raw datasets into selected models and reusable artifacts. The inference side accepts user inputs and routes them through the same transformations before prediction.

5.3 System Architecture

Overall System Architecture

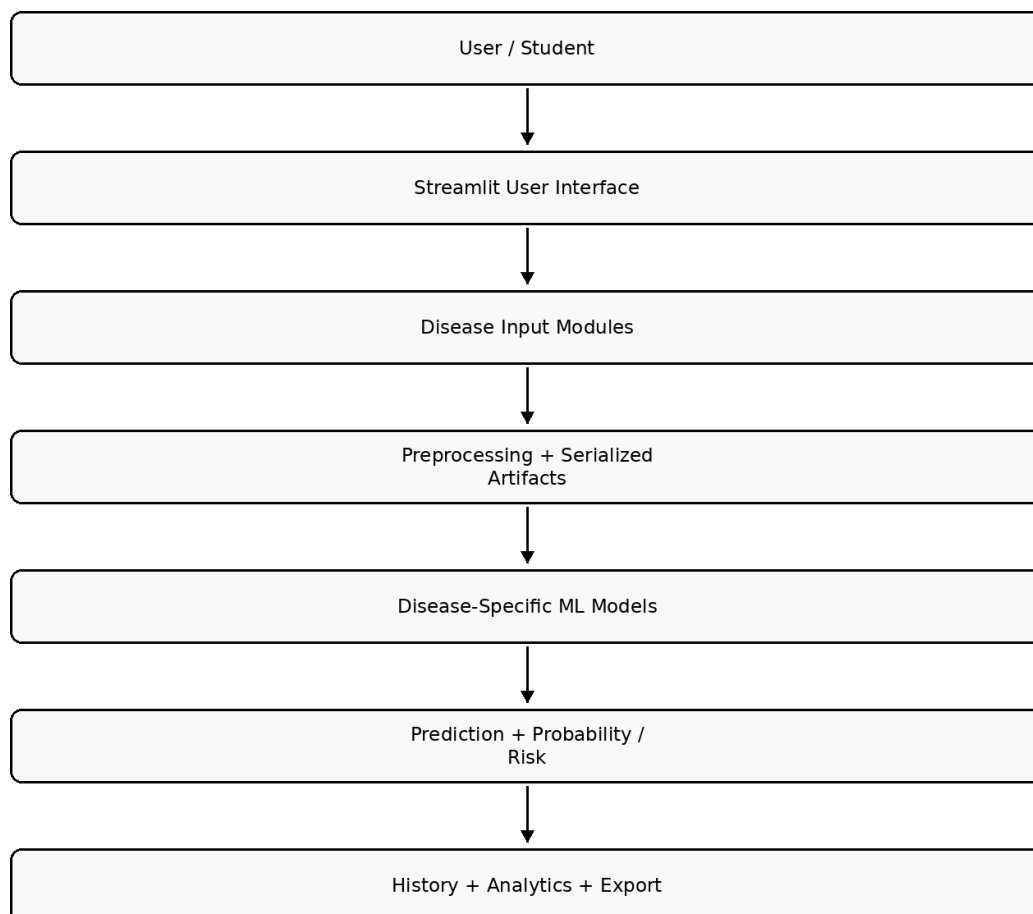


Figure 5.1 - Overall System Architecture

The architecture separates user interaction, input modules, preprocessing artifacts, model inference, and persistence/analytics concerns. This separation allows the model training process to remain independent of individual prediction sessions.

5.4 Overall System Workflow

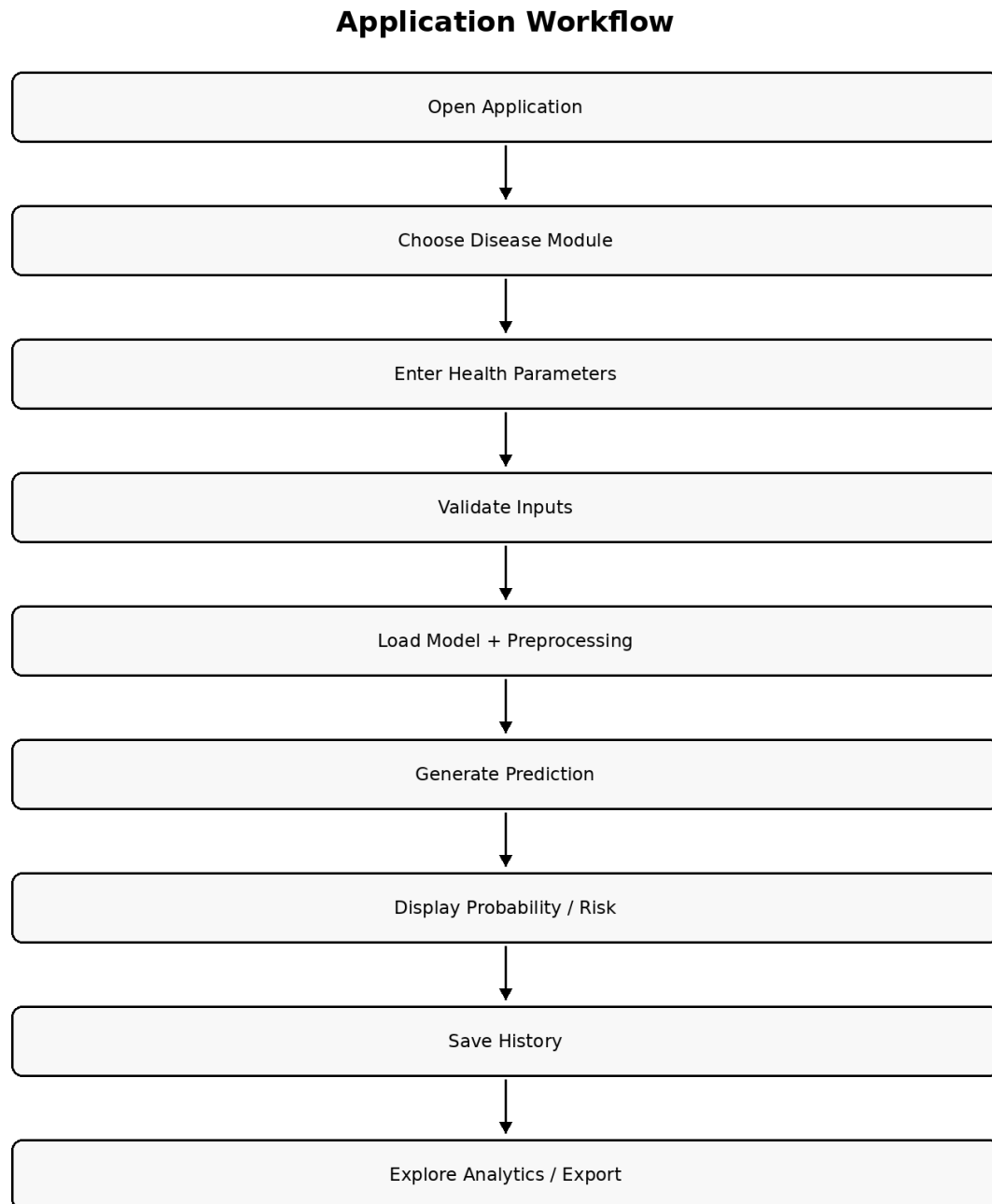


Figure 5.2 - Application Workflow

5.5 Machine Learning Pipeline

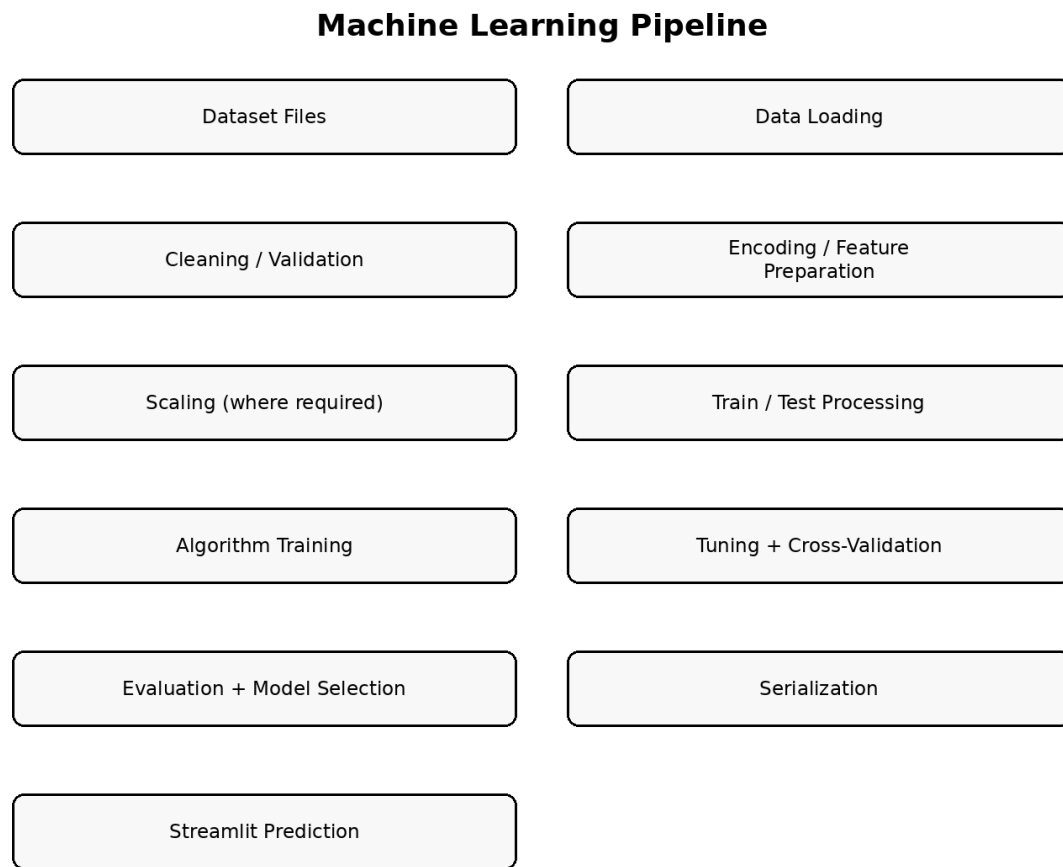


Figure 5.3 - Machine Learning Pipeline

The workflow is: Dataset -> Data Loading -> Data Preprocessing -> Data Cleaning / Validation -> Encoding -> Feature Preparation -> Scaling where required -> Train/Test Processing -> Multiple Algorithm Training -> Model Comparison -> Hyperparameter Tuning -> Cross-Validation -> Performance Evaluation -> Best Model Selection -> Model Serialization -> Streamlit Prediction -> Probability / Risk Interpretation -> Prediction History -> Analytics / Export.

5.6 Application Architecture

Application Module Architecture

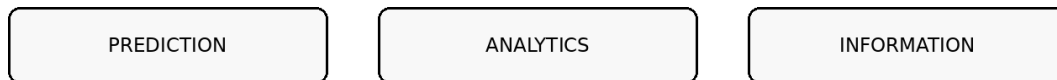


Figure 5.4 - Module Architecture

5.7 UML Diagrams

The diagrams in this report are intentionally high-level because the supplied source code was not provided as part of the report-generation inputs. The final technical implementation may refine individual interactions while preserving the documented responsibilities.

5.8 Use Case Diagram

Use Case Overview

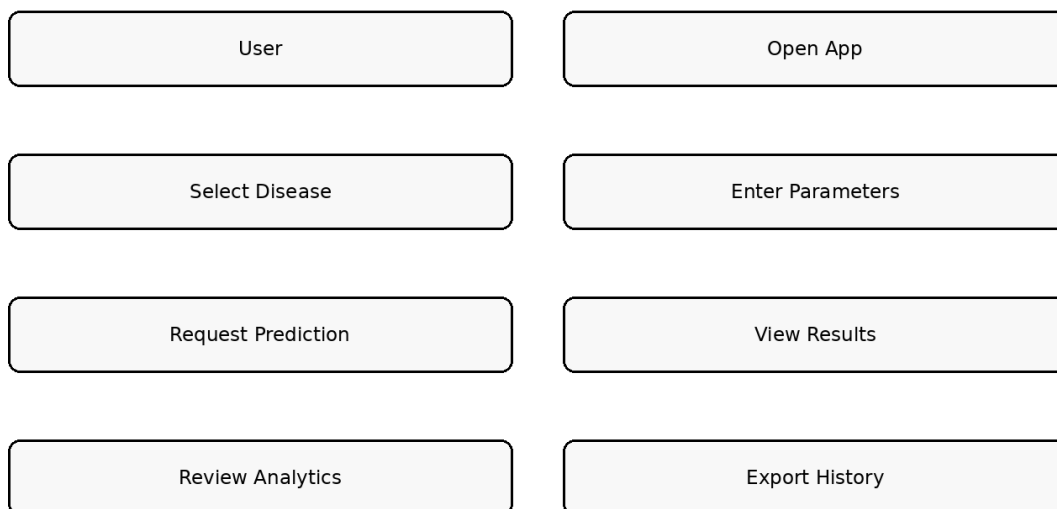


Figure 5.5 - Use Case Overview

5.9 Activity Diagram

Activity Flow

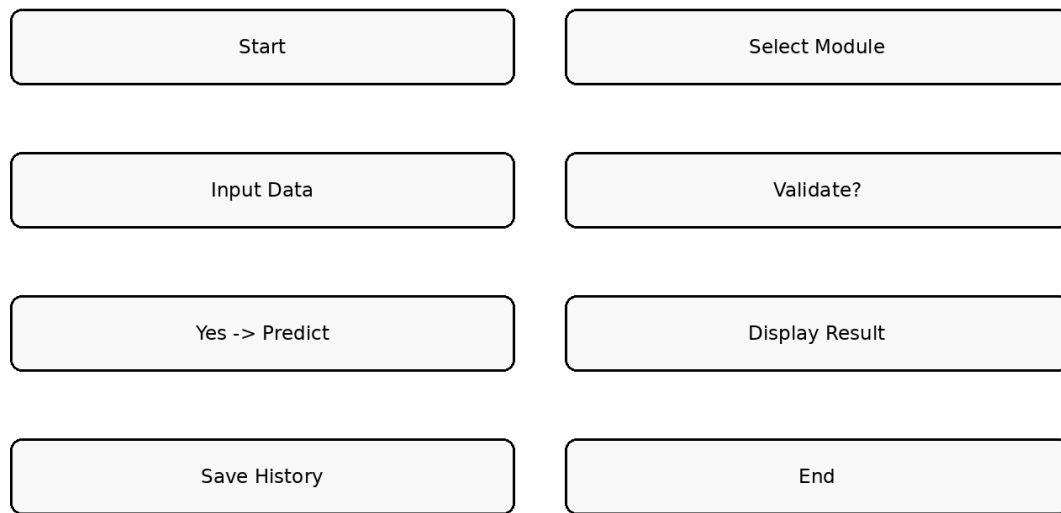


Figure 5.6 - Activity Flow

5.10 Sequence Diagram

Sequence Overview

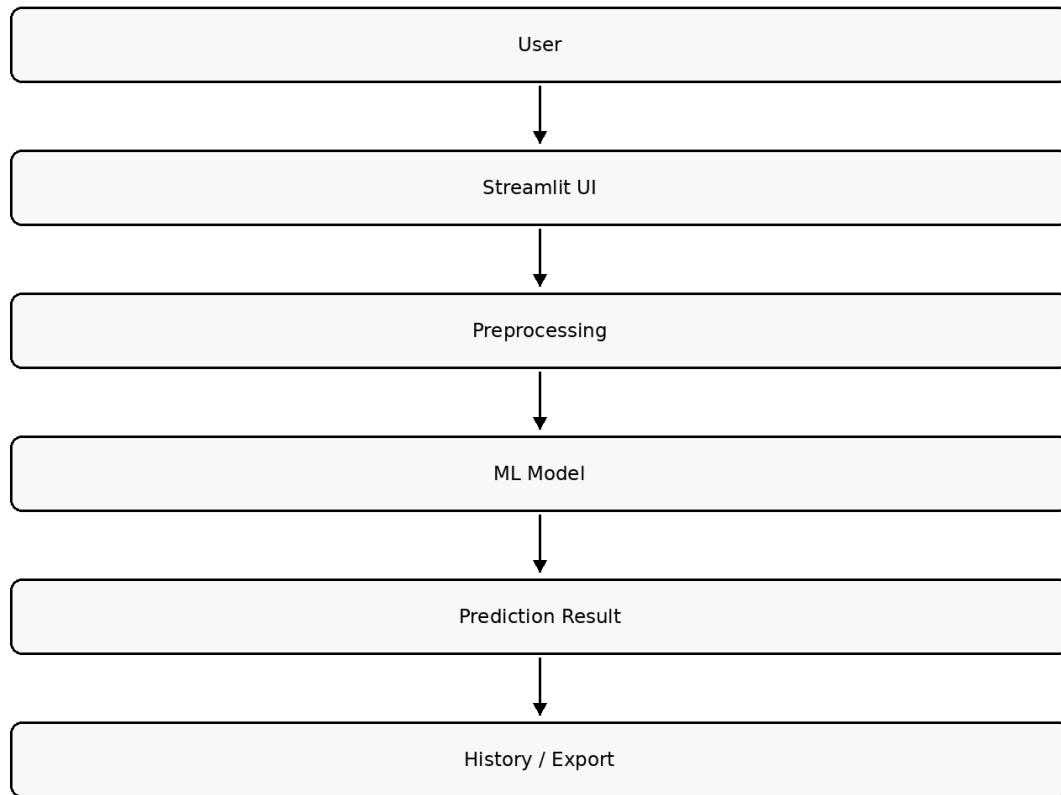


Figure 5.7 - Prediction Sequence Overview

5.11 Class / Component Diagram

Application Module Architecture

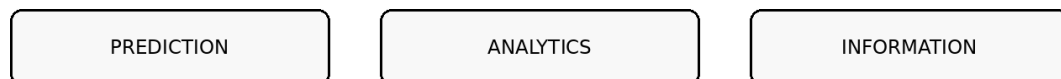


Figure 5.8 - Component-Oriented Module View

5.12 Data Flow Diagram

DFD Level 0

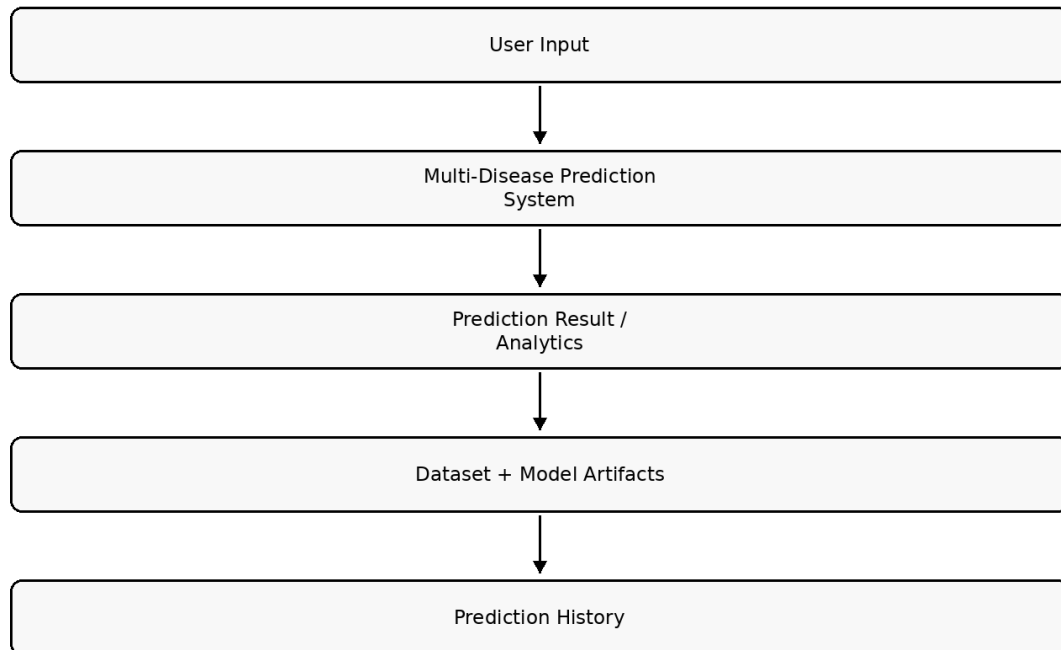


Figure 5.9 - Data Flow Diagram Level 0

DFD Level 1

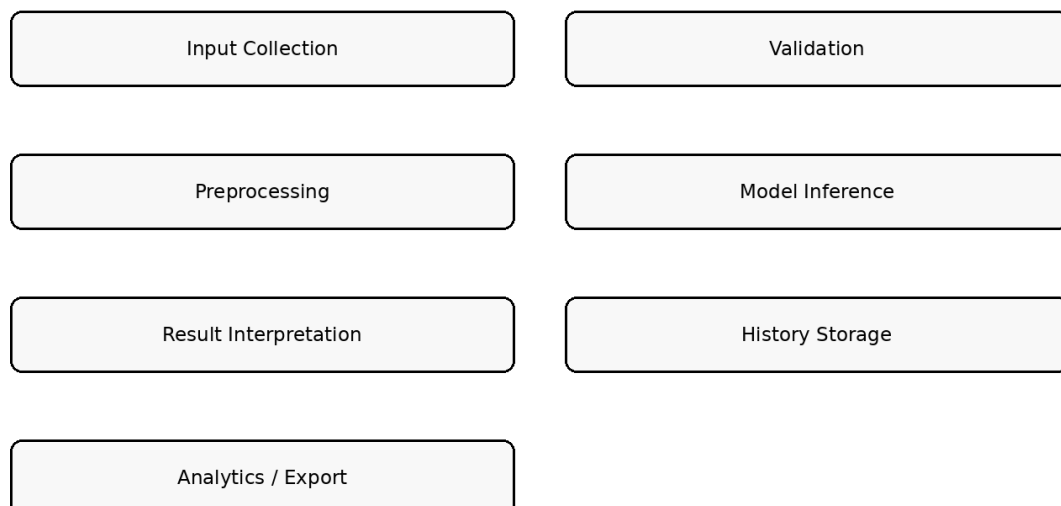


Figure 5.10 - Data Flow Diagram Level 1

5.13 Module Interaction

Application Navigation Hierarchy

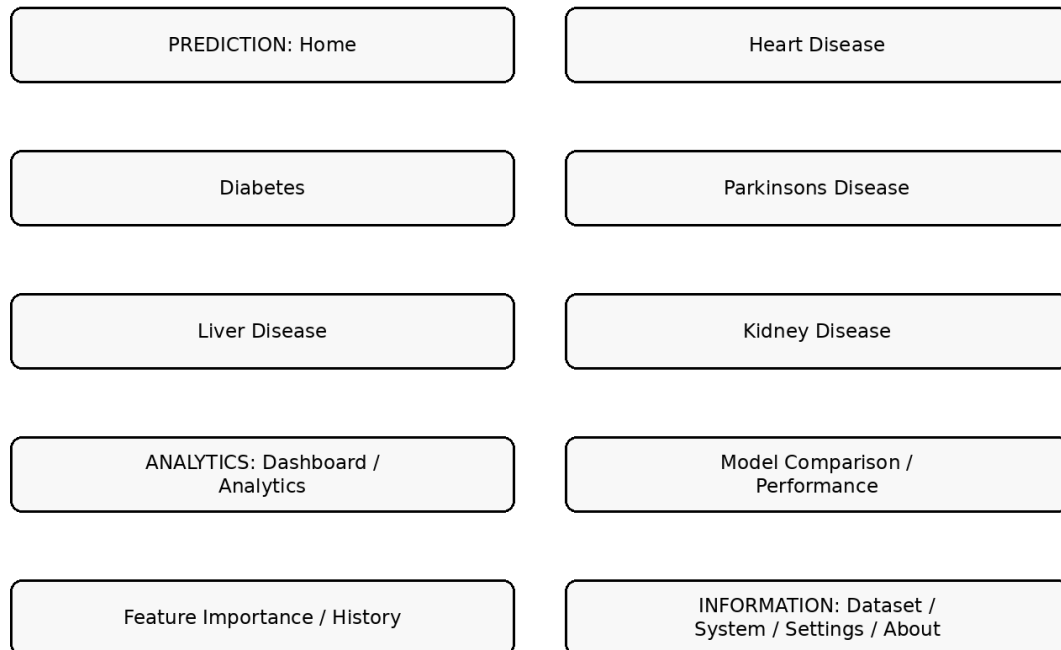


Figure 5.11 - Application Navigation Hierarchy

CHAPTER 6 - DATASET AND DATABASE / DATA DESIGN

6.1 Data Design Overview

The project is primarily file- and artifact-oriented rather than database-oriented. It uses five dataset files for training, serialized machine-learning models and preprocessing artifacts for inference, a CSV file for prediction history, and a JSON file for user settings. A relational database is not claimed or introduced where none is documented in the supplied project information.

6.2 Dataset Architecture

Each disease has an independent dataset and model pipeline. This design is necessary because the feature names, feature counts, data types, encoding requirements, and target variables differ between diseases.

6.3 Dataset Description

Disease	Dataset File	Major Features	Target	Purpose
Heart Disease	heart.csv	Demographics, cardiovascular symptoms, blood pressure, cholesterol, ECG and exercise variables	HeartDisease	Train heart-disease classification model
Diabetes	diabetes.csv	Demographics, hypertension, heart disease, smoking history, BMI, HbA1c, glucose	diabetes	Train diabetes classification model
Parkinson's Disease	parkinsons.data	22 voice/biomedical measurements	status	Train Parkinson's classification model
Liver Disease	Liver.csv	Age, gender, bilirubin, liver enzymes, proteins and biochemical variables	Dataset	Train liver-disease classification model
Kidney Disease	kidney.csv	Blood pressure, albumin, sugar, glucose, urea, creatinine, sodium, potassium, hemoglobin and related parameters	classification	Train chronic-kidney-disease classification model

6.4 Heart Disease Dataset

The heart dataset uses cardiovascular and demographic attributes such as age, sex, chest-pain category, resting blood pressure, cholesterol, fasting blood sugar, resting ECG, maximum heart rate, exercise-induced angina, oldpeak, and ST slope. The target column is HeartDisease.

6.5 Diabetes Dataset

The diabetes dataset contains demographic and health-related features including gender, age, hypertension, heart disease, smoking history, BMI, HbA1c level, and blood glucose level. The target column is diabetes.

6.6 Parkinson's Dataset

The Parkinson's dataset contains 22 biomedical/voice features. The target status indicates the labelled classification outcome used for supervised learning.

6.7 Liver Disease Dataset

The liver dataset contains age, gender, bilirubin measures, liver enzymes, protein-related attributes, and other biochemical variables. The supplied project information identifies the target column as Dataset.

6.8 Kidney Disease Dataset

The kidney dataset contains laboratory-style and clinical parameters including blood pressure, specific gravity, albumin, sugar, blood glucose, blood urea, serum creatinine, sodium, potassium, hemoglobin, and other kidney-related parameters. The target column is classification.

6.9 Feature Representation

Features are represented according to the source dataset. Numerical variables remain numerical, while categorical variables require encoding consistent with the training pipeline. The exact transformations must be preserved so that the model receives the same feature representation during inference.

6.10 Target Variables

Disease	Target Column	Interpretation in Application
Heart Disease	HeartDisease	Binary classification output
Diabetes	diabetes	Binary classification output

Disease	Target Column	Interpretation in Application
Parkinson's Disease	status	Binary classification output
Liver Disease	Dataset	Binary classification output
Kidney Disease	classification	Binary classification output

6.11 Data Preprocessing

Data preprocessing is an important step in the machine-learning workflow because the quality and consistency of input data directly affect model performance. In this project, preprocessing includes data validation, handling missing or inconsistent values, encoding categorical data where required, preparing the required features, and applying scaling when necessary. The same preprocessing steps used during model training are maintained during prediction to ensure consistency. These transformations are coupled with the corresponding model artifacts so that new user inputs are processed using the same feature representation as the training data.

6.12 Encoding

Encoding is performed when the dataset contains categorical values that cannot be directly processed by machine-learning algorithms. These categorical values are converted into suitable numerical representations using appropriate encoding techniques. The encoder used during training is saved as an artifact and reused during prediction. This ensures that user-provided categorical values are transformed consistently and prevents the system from creating a different encoding based on individual input data.

6.13 Scaling

Scaling is applied when required by the selected machine-learning algorithm or when the input features have significantly different numerical ranges. It transforms the feature values into a suitable scale while maintaining their relative relationships. The scaler parameters obtained during training are serialized along with the model artifacts. During prediction, the same scaler is applied to user-provided values, ensuring that the input data follows the same scale and distribution expected by the trained model.

6.14 Feature Preparation

Feature preparation involves identifying the required input variables for each disease prediction model and arranging them in the correct order. Since each disease model may require a different set of features, the application maintains model-specific feature information.

The `feature_info.pkl` artifact is used to store the expected feature details, helping the system validate and organize user inputs before prediction. This ensures that the model receives the correct number, type, and order of features and reduces the possibility of prediction errors caused by incorrect input mapping.

6.15 Model Artifact Storage

The `models` directory is used to store the trained machine-learning model files along with the related preprocessing artifacts such as scalers, encoders, and feature information. Storing these artifacts allows the application to directly load the required model during prediction instead of retraining it every time the application is executed. This reduces runtime processing and improves the overall efficiency of the system. It also preserves the trained model state and ensures that the same preprocessing configuration is used during inference.

6.16 Prediction History Storage

The prediction history is maintained in the `prediction_history.csv` file. It acts as a lightweight storage mechanism for recording prediction-related activities performed through the application. The stored information can be used to review previous predictions and provides useful functionality such as searching, filtering, and exporting historical records. This approach provides a simple and efficient method for maintaining application-level prediction records without requiring a separate database system.

6.17 Export Data Structure

The export functionality allows users to save prediction history and application results in different formats for further reference and analysis. **CSV and Excel formats** provide structured tabular data that can be easily opened and analysed using spreadsheet applications. **PDF format** provides a more presentation-oriented representation of the results and can be used for documentation or reporting purposes. These export options improve the usability of the system by allowing generated results to be stored, reviewed, and shared when required.

CHAPTER 7 - LIMITATIONS

7.1 Project Limitations

The developed application is primarily an academic and educational machine-learning project designed to demonstrate the use of classification models on healthcare datasets. It is not intended to function as a production-level clinical system. The predictions generated by the application should not be considered as medical diagnoses or used for making clinical treatment decisions.

7.2 Dataset Limitations

The performance of the developed models depends significantly on the quality and characteristics of the datasets used for training. Factors such as missing values, incomplete records, limited sample sizes, dataset bias, and variations in data collection methods can affect model performance. In addition, the datasets may not represent all population groups or healthcare environments, which can limit the reliability of the models when applied to different populations.

7.3 Model Limitations

Machine-learning models learn patterns from the data used during training and therefore their performance depends on the quality and distribution of that data. Even if a model achieves good evaluation results on testing data, its performance may vary when applied to completely new or unseen datasets. Model performance can also be influenced by the selected algorithm, preprocessing techniques, feature definitions, and quality of input data.

7.4 Prediction Limitations

The prediction generated by the system represents the output of a trained machine-learning model based on the input features provided by the user. A prediction probability should not automatically be interpreted as the actual probability of disease for an individual unless the model has been properly calibrated and clinically validated. Therefore, the results generated by this application are intended only for educational and research purposes.

7.5 Generalization Limitations

The trained models may not perform equally well across different age groups, geographical regions, population characteristics, healthcare environments, measurement devices, or clinical settings. Differences between the training data and real-world data can affect prediction

performance. Independent external validation using datasets from different sources would therefore be required before applying the models to broader populations.

7.6 Medical Interpretation Limitations

The prediction results provided by the application are not a replacement for professional medical advice, diagnosis, or treatment. The system is intended to demonstrate machine-learning concepts rather than provide clinical recommendations. Users experiencing health-related symptoms or requiring medical decisions should consult a qualified healthcare professional

7.7 Future Improvement Opportunities

The project can be further improved through additional validation, monitoring, and model-development techniques. Possible future improvements include:

- Evaluating the trained models using independent external datasets.
- Performing calibration analysis to improve the interpretation of prediction probabilities.
- Selecting clinically meaningful decision thresholds where appropriate.
- Introducing advanced explainability techniques to provide better understanding of model predictions.
- Implementing systematic data-quality monitoring and schema validation.
- Expanding the datasets with more diverse and representative samples.
- Periodically evaluating and retraining models when new high-quality data becomes available.

CHAPTER 8 - IMPLEMENTATION

8.1 Implementation Overview

Implementation proceeds from dataset preparation and model training to serialization and Streamlit integration. The project separates model creation from application inference so that users do not need to retrain models each time they use the application.

8.2 Environment Setup

1. Install Python 3.10+ and create an isolated environment.
2. Install the dependencies listed in requirements.txt.
3. Place the five datasets in the DataSets directory.
4. Ensure the trained model and preprocessing artifact files are available in models/.

8.3 Dataset Preparation

The datasets are loaded and validated within train_models.py. The preprocessing workflow prepares each disease dataset according to its feature types and training requirements. Because exact source-code details were not supplied, implementation-level statements are intentionally limited to the documented project behavior.

8.4 Model Training

The train_models.py script loads all five datasets, performs preprocessing, trains multiple candidate algorithms, applies hyperparameter tuning and cross-validation, evaluates the resulting models, selects the best-performing model for each disease, saves trained models and preprocessing artifacts, and generates evaluation reports.

8.5 Model Evaluation

Evaluation includes Accuracy, Precision, Recall, F1 Score, ROC-AUC, and Confusion Matrix. These metrics provide complementary views of classification quality. The report does not fabricate numerical values that were not supplied.

8.6 Model Serialization

Pickle-based serialization stores trained models and preprocessing objects for reuse. The models folder includes disease-specific model files, scaler files, encoder files, and feature_info.pkl.

8.7 Streamlit Application Development

The Streamlit application exposes navigation, form inputs, prediction results, dashboards, analytics, model comparison, feature importance, history, information pages, and settings. Streamlit is used as the application framework because it supports interactive Python-first data applications with comparatively little frontend boilerplate.

8.8 Prediction Module Implementation

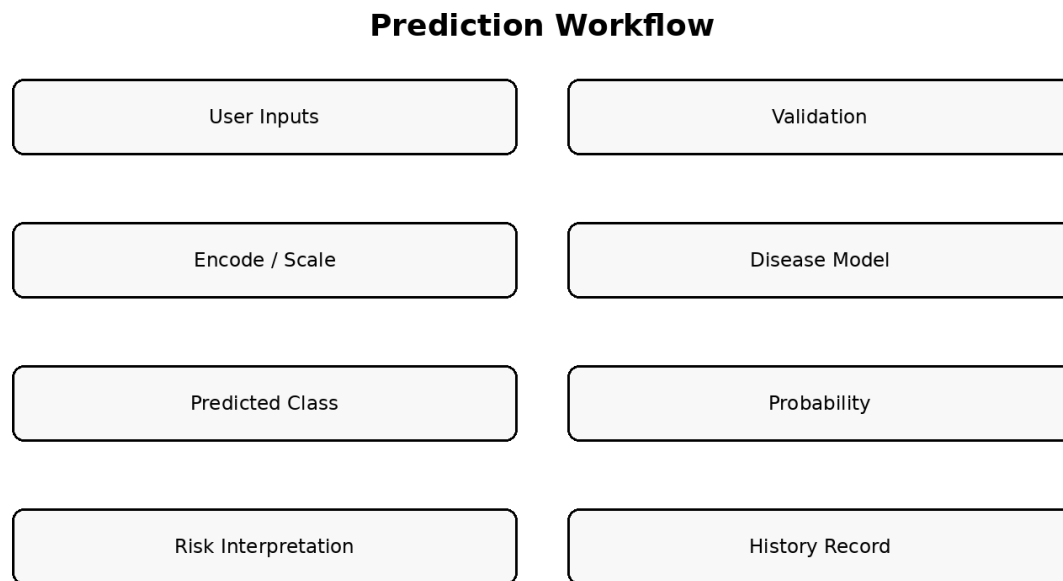


Figure 8.1 - Prediction Workflow

8.9 Analytics Implementation

Analytics are generated from stored history and model-evaluation information. The documented outputs include disease distribution, detection rate, probability distributions, prediction timelines, metric visualizations, and radar charts.

8.10 Model Comparison Implementation

Candidate algorithms are evaluated within a consistent workflow so that the same decision process is used for each disease. The best-performing model is then saved for inference.

8.11 Prediction History

Prediction history is saved automatically in `prediction_history.csv`. Search and filter functionality is provided through the application interface, and selected history data can be exported.

8.12 Report Export

CSV and Excel exports use tabular data-processing capabilities. PDF generation uses fpdf2. Exporting allows the application to produce a portable representation of results or history.

8.13 Theme / Settings

The theme preference is persisted through user_settings.json. The settings module provides a simple example of local user-preference storage without introducing a database dependency.

8.14 Testing

Testing includes model loading validation, prediction consistency checks, input validation, model artifact verification, application-level checks, basic unit testing, and error-handling verification. The project identifies test_models.py, test_fix.py, and verify_models.py as testing/verification files.

8.15 Deployment

For local execution, the documented commands are:

```
python train_models.py
python verify_models.py
python test_models.py
streamlit run app.py
```

The deployed application is available at <https://organsenseai.streamlit.app/> . The deployment should still be described as an educational application and not as a clinical diagnostic service.

CHAPTER 9 - SOURCE CODE / IMPORTANT IMPLEMENTATION

This chapter documents the key implementation responsibilities without reproducing the full source tree. Exact source code is not included because the source files themselves were not supplied as report-generation inputs. The placeholders below should be replaced with verified code excerpts from the final GitHub repository when submitting a code-inclusive edition.

9.1 Application Entry Point

```
```python
st.set_page_config(
 page_title=APP_NAME,
 page_icon="\U0001F3E5",
 layout="wide",
 initial_sidebar_state="expanded"
)

=====
MAIN APPLICATION
=====

def main():
 """Main application entry point."""

 # Render sidebar
 render_sidebar()

 # Apply CSS
```

```
apply_custom_css()

Page routing

page = st.session_state.get('current_page', 'Home')

if page == "Home":
 home_page()
elif page in DISEASES:
 prediction_page(page)
elif page == "Dashboard":
 dashboard_page()
elif page == "Analytics":
 analytics_page()
elif page == "Model Comparison":
 model_comparison_page()
elif page == "Performance Dashboard":
 performance_dashboard_page()
elif page == "Feature Importance":
 feature_importance_page()
elif page == "Prediction History":
 prediction_history_page()
elif page == "Dataset Information":
 dataset_info_page()
elif page == "System Information":
 system_info_page()
elif page == "Settings":
```

```

 settings_page()

 elif page == "About":

 about_page()

Footer

render_footer()

if __name__ == '__main__':

 main()

'''

```

## 9.2 Dataset Loading

```

def load_dataset(filename):

 """Load a dataset from the datasets folder."""

 path = os.path.join(DATASET_DIR, filename)

 if not os.path.exists(path):

 raise FileNotFoundError(f"Dataset not found at {path}")

 if filename.endswith('.data'):

 df = pd.read_csv(path)

 else:

 df = pd.read_csv(path)

 print(f"\n{'='*60}")

 print(f"Loaded {filename}: {df.shape[0]} rows, {df.shape[1]} columns")

```

```
print(f'{'='*60}')
```

```
return df
```

### 9.3 Data Preprocessing

```
def preprocess_heart(df):
```

```
 """Preprocess Heart Disease dataset with advanced cleaning."""
```

```
 print("\nPreprocessing Heart Disease data...")
```

```
 data = df.copy()
```

```
 # Initial info
```

```
 print(f" Initial shape: {data.shape}")
```

```
 print(f" Missing values: {data.isnull().sum().sum()}")
```

```
 # Check for duplicates
```

```
 duplicates = data.duplicated().sum()
```

```
 if duplicates > 0:
```

```
 print(f" Removing {duplicates} duplicate rows")
```

```
 data = data.drop_duplicates()
```

```
 # Handle Cholesterol=0 (invalid values)
```

```
 zero_chol = (data['Cholesterol'] == 0).sum()
```

```
 if zero_chol > 0:
```

```
 print(f" Found {zero_chol} rows with Cholesterol=0 (invalid)")
```

```
 # Replace with median of non-zero values
```

```
 median_chol = data[data['Cholesterol'] > 0]['Cholesterol'].median()
```

```

data.loc[data['Cholesterol'] == 0, 'Cholesterol'] = median_chol

print(f" Replaced with median: {median_chol}")

Handle RestingBP=0 (invalid)

zero_bp = (data['RestingBP'] == 0).sum()

if zero_bp > 0:

 print(f" Found {zero_bp} rows with RestingBP=0 (invalid)")

 median_bp = data[data['RestingBP'] > 0]['RestingBP'].median()

 data.loc[data['RestingBP'] == 0, 'RestingBP'] = median_bp

Cap outliers for numerical columns

num_cols = ['Age', 'RestingBP', 'Cholesterol', 'MaxHR', 'Oldpeak']

for col in num_cols:

 if col in data.columns:

 outlier_count = detect_outliers_iqr(data, col).sum()

 if outlier_count > 0:

 data = cap_outliers(data, col)

 print(f" Capped {outlier_count} outliers in {col}")

Encode categorical variables

le_dict = {}

cat_columns = ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']

for col in cat_columns:

 if col in data.columns:

 le = LabelEncoder()

```

```

 data[col] = le.fit_transform(data[col].astype(str))

 le_dict[col] = le

Handle any remaining missing values
data = data.dropna()

Target column
target_col = 'HeartDisease'

if target_col not in data.columns:

 raise ValueError(f"Target column '{target_col}' not found in heart dataset")

X = data.drop(columns=[target_col])
y = data[target_col]

print(f" Final features: {list(X.columns)}")
print(f" Final shape: {X.shape}")
print(f" Class distribution: {y.value_counts().to_dict()}")

return X, y, le_dict

```

## 9.4 Model Loading

```

python

def load_model(model_name):

 """Load a trained model from the models folder."""

 path = os.path.join(MODEL_DIR, model_name)

 if not os.path.exists(path):

```

```

 st.error(f"⚠️ Model file not found: `{path}`. Please run `train_models.py`
first.")

 return None

 with open(path, 'rb') as f:

 return pickle.load(f)

def load_scaler(disease_key):

 """Load scaler for a disease."""

 path = os.path.join(MODEL_DIR, f'{disease_key}_scaler.pkl')

 if not os.path.exists(path):

 return None

 with open(path, 'rb') as f:

 return pickle.load(f)

def load_encoders(disease_key):

 """Load label encoders for a disease."""

 path = os.path.join(MODEL_DIR, f'{disease_key}_encoders.pkl')

 if not os.path.exists(path):

 return {}

 with open(path, 'rb') as f:

 return pickle.load(f)

def load_feature_info():

 """Load feature information for all diseases."""

 path = os.path.join(MODEL_DIR, 'feature_info.pkl')

```



```

if not os.path.exists(path):

 st.error("⚠ Feature info not found. Please run train_models.py first.") return

 None

with open(path, 'rb') as f:

 return pickle.load(f)

```

## 9.5 Prediction Function

```

def predict_disease(model, features, disease_key, feature_names, all_feature_names):

 """Make a prediction using the loaded model with preprocessing."""

 try:

 scaler = load_scaler(disease_key)

 feature_info = load_feature_info()

 # Build a DataFrame with all features

 feature_dict = dict(zip(feature_names, features))

 full_df = pd.DataFrame([feature_dict])

 # Scale features

 if scaler is not None:

 full_df = pd.DataFrame(scaler.transform(full_df), columns=full_df.columns)

 # Select features if feature selection was used

 if feature_info and disease_key in feature_info:

 selected_features = feature_info[disease_key].get('features', [])

```

```

 if selected_features:
 full_df = full_df[selected_features]

 prediction = model.predict(full_df.values)

 if hasattr(model, 'predict_proba'):
 probability = model.predict_proba(full_df.values)
 return prediction[0], np.max(probability)
 else:
 return prediction[0], 0.0

except Exception as e:
 st.error(f"Prediction error: {str(e)}")

import traceback
traceback.print_exc()

return None, None

```

## 9.6 Input Validation

```

def heart_disease_interface():
 """Display input form for Heart Disease prediction."""

 st.header("❤️ Heart Disease Prediction")

 with st.expander("❗ About Heart Disease Prediction", expanded=False):
 st.markdown("""
 This model predicts the risk of heart disease based on clinical parameters
 including
 age, cholesterol levels, blood pressure, ECG results, and exercise capacity.

 Enter the values below and click Predict to assess your risk.
 """)

```

```
""")
```

```
col1, col2, col3 = st.columns(3)
```

```
with col1:
```

```
 age = st.number_input("Age", min_value=1, max_value=120, value=30, step=1,
```

```
 help="Patient's age in years")
```

```
 sex = st.selectbox("Sex", ["Male", "Female"])
```

```
 chest_pain = st.selectbox(
```

```
 "Chest Pain Type",
```

```
 ["ATA (Atypical Angina)", "NAP (Non-Anginal Pain)",
```

```
 "ASY (Asymptomatic)", "TA (Typical Angina)"],
```

```
)
```

## 9.7 Risk Interpretation

```
python
```

```
def get_risk_level(probability, prediction):
```

```
 """Determine risk level based on probability."""
```

```
 if prediction == 0:
```

```
 return "Low Risk", "success", 0.0
```

```
 elif probability >= 0.75:
```

```
 return "High Risk", "error", probability
```

```
 elif probability >= 0.5:
```

```
 return "Moderate Risk", "warning", probability
```

```

else:

 return "Low Risk", "success", probability

def get_recommendations(disease, risk_level):

 """Generate health recommendations based on disease and risk level."""

 recommendations = {

 "Heart Disease": {

 "Low Risk": [

 "✔ Maintain a balanced diet rich in fruits and vegetables",

 "✔ Exercise at least 30 minutes daily",

 "✔ Regular health check-ups annually",

 "✔ Avoid smoking and limit alcohol intake"

],

 "Moderate Risk": [

 "⚠ Reduce sodium and saturated fat intake",

 "⚠ Increase cardiovascular exercise (walking, jogging, cycling)", "⚠

 Monitor blood pressure and cholesterol levels regularly", "⚠

 Consult a healthcare provider for a cardiac assessment",

 "⚠ Maintain a healthy BMI (18.5–24.9)"

],

 "High Risk": [

 "⦿ Consult a cardiologist immediately",

 "⦿ Reduce cholesterol through diet and prescribed medication",

```

```

 "🧘 Exercise regularly under medical guidance",
 "🚭 Strictly avoid smoking and alcohol",
 "🩺 Monitor blood pressure daily",
 "🧘 Consider stress management techniques (yoga, meditation)"
]

},

... (Diabetes, Parkinson's, Liver and Kidney dictionaries omitted for brevity)

}

return recommendations.get(disease, {}).get(risk_level, [])

```

## 9.8 Analytics

python

```
def analytics_page():
```

```
 """Render the Analytics page with charts and visualizations."""
```

```
 st.markdown('<h1 class="main-header">☑ Analytics & Visualizations</h1>',
 unsafe_allow_html=True)
```

```
 history = st.session_state.prediction_history
```

```
 if not history:
```

```
 st.warning("⚠ No prediction data available. Make some predictions to see analytics!")
```

```
 return
```

```
 df = pd.DataFrame(history)
```

```
 df['timestamp_dt'] = pd.to_datetime(df['timestamp'])
```

```
tab1, tab2, tab3, tab4 = st.tabs(["📊 Charts", "📈 Trends", "🔄 Distributions", "📋
Summary Stats"])
```

```
with tab1:
```

```
 # Disease distribution pie chart
```

```
 st.markdown("#### Disease Distribution")
```

```
 disease_counts = df['disease'].value_counts()
```

```
 fig = px.pie(
 values=disease_counts.values,
 names=disease_counts.index,
 color=disease_counts.index,
 color_discrete_map=DISEASE_COLORS,
 hole=0.3
)
```

```
 fig.update_traces(textposition='outside', textinfo='percent+label')
```

```
 fig.update_layout(height=400)
```

```
 st.plotly_chart(fig, use_container_width=True)
```

```
 # Detection Rate by Disease
```

```
 st.markdown("#### Detection Rate by Disease")
```

```
 outcome_by_disease = df.groupby(['disease',
'prediction']).size().reset_index(name='count')
```

```
 fig = px.bar(
 outcome_by_disease, x='disease', y='count', color='prediction',
 barmode='group',
```

```

 color_discrete_map={'Detected': '#FF6B6B', 'Not Detected': '#2ECC71'},
 labels={'disease': 'Disease', 'count': 'Count', 'prediction': 'Outcome'}
)
 fig.update_layout(height=400)
 st.plotly_chart(fig, use_container_width=True)

```

with tab2:

```

Prediction timeline

st.markdown("#### Prediction Timeline")

daily_counts = df.groupby('date').size().reset_index(name='count')

daily_counts['date'] = pd.to_datetime(daily_counts['date'])

daily_counts = daily_counts.sort_values('date')

fig = go.Figure()

fig.add_trace(go.Scatter(
 x=daily_counts['date'], y=daily_counts['count'],
 mode='lines+markers', name='Predictions',
 line=dict(color='#0077b6', width=3),
 marker=dict(size=8, color='#0077b6')
))

fig.update_layout(
 title="Daily Prediction Activity",
 xaxis_title="Date", yaxis_title="Number of Predictions", height=400
)

st.plotly_chart(fig, use_container_width=True)

```

## 9.9 Model Comparison

python

```
def compare_and_select_best(X_train, y_train, X_test, y_test, disease_name,
n_iter_search=10):

 """Compare multiple algorithms and select the best one using RandomizedSearchCV."""

 algorithms = get_algorithm_comparison()

 best_overall_model = None

 best_overall_score = -1

 best_overall_name = None

 best_overall_params = None

 comparison_results = []

 print(f'\n{'='*50}')

 print(f'Algorithm Comparison for {disease_name}')

 print(f'{'='*50}')

 for algo_name, algo_config in algorithms.items():

 print(f'\n Testing: {algo_name}...')

 start_time = time.time()

 try:

 # Use RandomizedSearchCV for efficiency

 random_search = RandomizedSearchCV(

 algo_config['model'],

 algo_config['params'],

 n_iter=min(n_iter_search, 10),
```



```

 cv=3,
 scoring='f1',
 n_jobs=-1,
 random_state=42,
 verbose=0
)

 random_search.fit(X_train, y_train)

 best_model = random_search.best_estimator_
 best_params = random_search.best_params_

 # Evaluate on test set
 y_pred = best_model.predict(X_test)
 y_prob = best_model.predict_proba(X_test)

 accuracy = accuracy_score(y_test, y_pred)
 precision = precision_score(y_test, y_pred, average='binary', zero_division=0)
 recall = recall_score(y_test, y_pred, average='binary', zero_division=0)
 f1 = f1_score(y_test, y_pred, average='binary', zero_division=0)

 # ROC-AUC
 roc_auc = None
 if len(np.unique(y_test)) > 1:
 try:
 roc_auc = roc_auc_score(y_test, y_prob[:, 1])

```

```

except Exception:
 pass

elapsed = time.time() - start_time

print(f" Accuracy: {accuracy:.4f}, F1: {f1:.4f}, ROC-AUC: {roc_auc}, Time:
{elapsed:.1f}s")

result = {
 'algorithm': algo_name,
 'accuracy': accuracy,
 'precision': precision,
 'recall': recall,
 'f1_score': f1,
 'roc_auc': roc_auc,
 'best_params': best_params,
 'training_time': elapsed
}
comparison_results.append(result)

Select based on F1 score (handles class imbalance well)
if f1 > best_overall_score:
 best_overall_score = f1
 best_overall_model = best_model
 best_overall_name = algo_name
 best_overall_params = best_params

```

```

except Exception as e:

 print(f" ERROR: {str(e)[:100]}")

 continue

print(f"\n {'='*40}")

print(f" BEST ALGORITHM: {best_overall_name}")

print(f" Best F1 Score: {best_overall_score:.4f}")

print(f" Best Params: {best_overall_params}")

print(f" {'='*40}")

return best_overall_model, best_overall_name, comparison_results

```

## 9.10 Prediction History

```

python

def add_prediction_record(disease, features, prediction, probability, risk_level,
patient_name="Unknown"):

 """Add a prediction record to history."""

 record = {

 'timestamp': datetime.now().strftime('%Y-%m-%d %H:%M:%S'),

 'date': datetime.now().strftime('%Y-%m-%d'),

 'disease': disease,

 'patient_name': patient_name,

 'prediction': 'Detected' if prediction == 1 else 'Not Detected',

 'probability': round(float(probability), 4),

 'risk_level': risk_level,

```

```

 'features': str(features)
 }

 st.session_state.prediction_history.append(record)

 save_prediction_history()

 st.session_state.prediction_made = True

 st.session_state.last_prediction = record

def save_prediction_history():
 """Save prediction history to CSV."""
 if not st.session_state.prediction_history:
 return

 df = pd.DataFrame(st.session_state.prediction_history)

 df.to_csv(HISTORY_FILE, index=False)

def load_prediction_history():
 """Load prediction history from CSV."""
 if os.path.exists(HISTORY_FILE):
 try:
 df = pd.read_csv(HISTORY_FILE)

 st.session_state.prediction_history = df.to_dict('records')

 except:
 st.session_state.prediction_history = []

def prediction_history_page():

```

```

"""Render the Prediction History page with search, filter, and export."""

st.markdown('<h1 class="main-header">📋 Prediction History</h1>',
unsafe_allow_html=True)

history = st.session_state.prediction_history

if not history:

 st.warning("⚠️ No prediction history available yet. Make some predictions first!")

 return

df = pd.DataFrame(history)

col1, col2, col3 = st.columns(3)

with col1:

 search_query = st.text_input("🔍 Search by patient name or disease", "")

with col2:

 filter_date = st.date_input("📅 Filter by date", value=None)

with col3:

 all_diseases = ["All"] + sorted(df['disease'].unique().tolist())

 filter_disease = st.selectbox("☐ Filter by disease", all_diseases)

Apply filters

filtered_df = df.copy()

```

```

if search_query:

 filtered_df = filtered_df[

 filtered_df['patient_name'].astype(str).str.contains(search_query, case=False,
na=False) |

 filtered_df['disease'].astype(str).str.contains(search_query, case=False,
na=False)

]

if filter_date:

 filter_date_str = filter_date.strftime('%Y-%m-%d')

 filtered_df = filtered_df[filtered_df['date'] == filter_date_str]

if filter_disease != "All":

 filtered_df = filtered_df[filtered_df['disease'] == filter_disease]

Display count

st.markdown(f"<div class='insight'>Showing {len(filtered_df)}
of {len(df)} records</div>",

 unsafe_allow_html=True)

if not filtered_df.empty:

 display_df = filtered_df[['timestamp', 'disease', 'patient_name', 'prediction',
'risk_level', 'probability']].copy()

 display_df['probability'] = display_df['probability'].apply(lambda x:
f"{float(x)*100:.1f}%")

```

```

display_df.columns = ['Timestamp', 'Disease', 'Patient', 'Result', 'Risk Level',
'Probability']

st.dataframe(display_df, use_container_width=True, height=400)

else:

 st.info("No records match your search/filter criteria.")

```

## 9.11 Report Export

python

```

def export_to_csv(records):

 """Export prediction history to CSV and return as string."""

 if not records:

 return None

 output = io.StringIO()

 fieldnames = ['timestamp', 'date', 'disease', 'patient_name', 'prediction', 'probability',
'risk_level']

 writer = csv.DictWriter(output, fieldnames=fieldnames)

 writer.writeheader()

 for r in records:

 writer.writerow({k: r.get(k, "") for k in fieldnames})

 return output.getvalue()

def export_to_pdf(records):

 """Export prediction history to PDF and return bytes."""

 if not records:

 return None

```

```

pdf = FPDF()
pdf.add_page()
pdf.set_font("Helvetica", "B", 16)
pdf.cell(0, 10, "Prediction History Report", ln=True, align='C')
pdf.set_font("Helvetica", "", 10)
pdf.cell(0, 8, f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}",
ln=True, align='C')
pdf.ln(5)

Column headers
pdf.set_font("Helvetica", "B", 9)
col_widths = [35, 35, 30, 30, 25, 25]
headers = ['Date', 'Disease', 'Patient', 'Prediction', 'Prob.', 'Risk']
for i, h in enumerate(headers):
 pdf.cell(col_widths[i], 7, h, border=1, align='C')
pdf.ln()

Records
pdf.set_font("Helvetica", "", 8)
for r in records[:100]: # Limit to 100 records for PDF
 vals = [
 r.get('date', " ")[-10:],
 r.get('disease', " ")[0:18],
 r.get('patient_name', " ")[0:12],
 r.get('prediction', " ")[0:8],
 str(round(float(r.get('probability', 0))*100, 1)) + '%',

```



```

 r.get('risk_level', "")[:8]
]
 for i, v in enumerate(vals):
 pdf.cell(col_widths[i], 6, v, border=1, align='C')
 pdf.ln()

Footer
pdf.ln(10)
pdf.set_font("Helvetica", "I", 8)
pdf.cell(0, 6, f"Total Records: {len(records)}", ln=True)
pdf.cell(0, 6, f"{APP_NAME} v{APP_VERSION}", ln=True)

return pdf.output(dest='S').encode('latin-1', errors='replace')

```

## 9.12 Testing

```

python

import pickle

import sys

sys.path.insert(0, 'c:/Users/raghu/Desktop/multi-disease prediction system')

from app import get_feature_importance, load_feature_info

Test Parkinson's
print("=" * 70)

print("PARKINSON'S DISEASE - Feature Importance Test")

```

```

print("=" * 70)

with open('models/parkinsons_model.pkl', 'rb') as f:

 parkinsons_model = pickle.load(f)

UI passes 22 features (what the form returns)
parkinsons_ui_features = ['MDVP:F0(Hz)', 'MDVP:Fhi(Hz)', 'MDVP:Flo(Hz)',
'MDVP:Jitter(%)',

 'MDVP:Jitter(Abs)', 'MDVP:RAP', 'MDVP:PPQ', 'Jitter:DDP',

 'MDVP:Shimmer', 'MDVP:Shimmer(dB)', 'Shimmer:APQ3',
'Shimmer:APQ5',

 'MDVP:APQ', 'Shimmer:DDA', 'NHR', 'HNR', 'RPDE', 'DFA',

 'spread1', 'spread2', 'D2', 'PPE']

feat_imp = get_feature_importance(parkinsons_model, parkinsons_ui_features,
disease_name="Parkinson's Disease")

if feat_imp is not None:

 print(f"✓ Feature importance extracted successfully!")

 print(f" Number of features: {len(feat_imp)}")

 print(f" Top 5: {list(feat_imp['Feature']).head()}")

else:

 print(f"✗ Feature importance returned None")

'''

```

### ### 9.12.3 verify\_models.py - Loaded-model consistency verification

```

```python

import pickle

import json

import pandas as pd

import numpy as np


# Load feature info

with open('models/feature_info.pkl', 'rb') as f:

    feature_info = pickle.load(f)


print("=" * 70)

print("FEATURE INFO VERIFICATION")

print("=" * 70)


for key, info in feature_info.items():

    print(f"\n{key.upper()}:")

    print(f" Algorithm: {info.get('algorithm')}")

    print(f" Accuracy: {info.get('accuracy')}")

    print(f" Selected features ({info.get('n_features')}): {info.get('features')}")


models = {

    'heart': 'models/heart_model.pkl',

    'diabetes': 'models/diabetes_model.pkl',

```

```

'parkinsons': 'models/parkinsons_model.pkl',

'liver': 'models/liver_model.pkl',

'kidney': 'models/kidney_model.pkl'

}

print("\n" + "=" * 70)

print("MODEL VERIFICATION")

print("=" * 70)


for key, path in models.items():

    with open(path, 'rb') as f:

        model = pickle.load(f)


    n_features_in = getattr(model, 'n_features_in_', 'N/A')

    has_feature_importances = hasattr(model, 'feature_importances_')

    feature_importances_len = len(model.feature_importances_) if
has_feature_importances else 'N/A'


    info = feature_info.get(key, {})

    selected_features = info.get('features', [])

    all_features = info.get('all_features', [])

```

CHAPTER 10 - CONCLUSION AND FUTURE WORK

10.1 Conclusion

The internship project successfully establishes a complete educational workflow for multi-disease machine-learning prediction. It combines five disease-specific datasets, multiple candidate classification algorithms, model evaluation, serialized artifacts, and a Streamlit interface within one coherent project.

10.2 Project Outcomes

- Created a unified application structure for five disease prediction tasks.
- Implemented a model-comparison mindset where model choice is disease-specific.
- Integrated model outputs with interactive Streamlit forms and analytics.
- Added history, dataset inspection, system information, settings, and export features.
- Produced a deployment-ready educational application at the supplied Streamlit address.

10.3 Learning Outcomes

- Practical understanding of supervised machine-learning classification.
- Experience with data preprocessing and feature representation.
- Exposure to hyperparameter tuning, cross-validation and model evaluation.
- Experience integrating machine-learning artifacts into a web application.
- Understanding of responsible communication of AI results in a healthcare-oriented context.
- Improved familiarity with testing, documentation, deployment, and technical reporting.

10.4 Future Enhancements

- Add external validation and calibration studies for stronger reliability evidence.
- Introduce a more formal model registry and versioning strategy.
- Add explainability dashboards with clearly documented methods.
- Improve privacy and secure handling of user history.
- Add automated schema checks and data-quality reports.
- Perform controlled domain validation before considering use beyond academic research.

10.5 Final Remarks

This project demonstrates the educational value of combining machine learning with interactive application development. Its strongest contribution is as a practical learning artifact that connects the full lifecycle from dataset to model to interface to evaluation. The system should remain clearly positioned as an academic/research project and not as a substitute for professional medical judgment.

The **Multi-Disease Prediction System** demonstrates the practical application of machine learning in a healthcare-oriented software application. The project covers important stages such as data preprocessing, model training, evaluation, prediction, and user interface development.

It provides practical experience in integrating **Python, machine learning, data science, and application development** into a single working system. The project also highlights the importance of model evaluation, proper data handling, and user-friendly design.

However, the system is intended strictly for **academic, educational, and research purposes**. Its predictions should not be considered a medical diagnosis or a replacement for professional medical advice.

In conclusion, the Multi-Disease Prediction System should primarily be regarded as a **practical academic and research-oriented project** that demonstrates the educational value of applying machine learning to a healthcare-related problem. Its greatest contribution is not the replacement of professional medical decision-making, but the demonstration of how datasets, predictive models, software interfaces, evaluation techniques, and deployment technologies can be integrated into a meaningful end-to-end application. The project therefore serves as a strong foundation for further learning, experimentation, and future development in the fields of **Artificial Intelligence, Machine Learning, Data Science, and Healthcare Technology**.

REFERENCES

- [1] Python Software Foundation. Python Documentation. <https://docs.python.org/3/>
 - [2] Streamlit. Streamlit Documentation. <https://docs.streamlit.io/>
 - [3] scikit-learn developers. scikit-learn 1.5 Documentation. <https://scikit-learn.org/1.5/>
 - [4] Pandas development team. pandas Documentation. <https://pandas.pydata.org/docs/>
 - [5] NumPy developers. NumPy Documentation. <https://numpy.org/doc/>
 - [6] Matplotlib developers. Matplotlib Documentation. <https://matplotlib.org/stable/>
 - [7] Seaborn developers. Seaborn Documentation. <https://seaborn.pydata.org/>
 - [8] Plotly. Plotly Python Documentation. <https://plotly.com/python/>
 - [9] fpdf2 project. fpdf2 Documentation. <https://py-pdf.github.io/fpdf2/>
 - [10] UCI Machine Learning Repository. Heart Disease / public machine-learning datasets. <https://archive.ics.uci.edu/>
 - [11] Rajan et al. / public dataset source. Parkinson's voice measurement dataset; verify exact source metadata before final academic citation. Public dataset source as identified in the project files
 - [12] Indian Liver Patient Dataset / public dataset source. Liver disease dataset; verify exact source metadata before final academic citation. Public dataset source as identified in the project files
 - [13] Chronic Kidney Disease / public dataset source. Kidney disease dataset; verify exact source metadata before final academic citation. Public dataset source as identified in the project files
 - [14] TVK Technologies. Internship Completion Letter dated 08 August 2026. Internal internship document supplied for report preparation
 - [15] Project repository. Multi-Disease Prediction System - GitHub Repository. <https://github.com/raghul692/Multi-Prediction-Sytem>
 - [16] Project deployment. OrganSenseAI Streamlit Application. <https://organsenseai.streamlit.app/>
- Reference note: exact publication metadata for the public datasets should be confirmed against the original dataset landing pages before submitting a version intended for formal bibliographic review. No unverified publication details have been invented in this report.

APPENDIX A - PROJECT STRUCTURE

The following structure documents the supplied project organization.

```
multi-disease prediction system/
|-- DataSets/
|   |-- heart.csv
|   |-- diabetes.csv
|   |-- parkinsons.data
|   |-- Liver.csv
|   `-- kidney.csv
|-- models/
|   |-- heart_model.pkl
|   |-- diabetes_model.pkl
|   |-- parkinsons_model.pkl
|   |-- liver_model.pkl
|   |-- kidney_model.pkl
|   |-- *_model_scaler.pkl
|   |-- *_model_encoders.pkl
|   `-- feature_info.pkl
|-- reports/
|   |-- generate_report.py
|   |-- model_evaluation_report.json
|   `-- model_evaluation_report.txt
|-- app.py
|-- train_models.py
|-- verify_models.py
|-- test_models.py
|-- test_fix.py
|-- requirements.txt
|-- prediction_history.csv
|-- user_settings.json
`-- README.md
```

Directory purpose: DataSets contains training inputs; models contains trained model and preprocessing artifacts; reports contains generated evaluation/reporting outputs; application and training scripts define runtime and model-building behavior; history/settings files provide local persistence; README.md documents the project for users and developers.

APPENDIX B - SAMPLE SCREENS

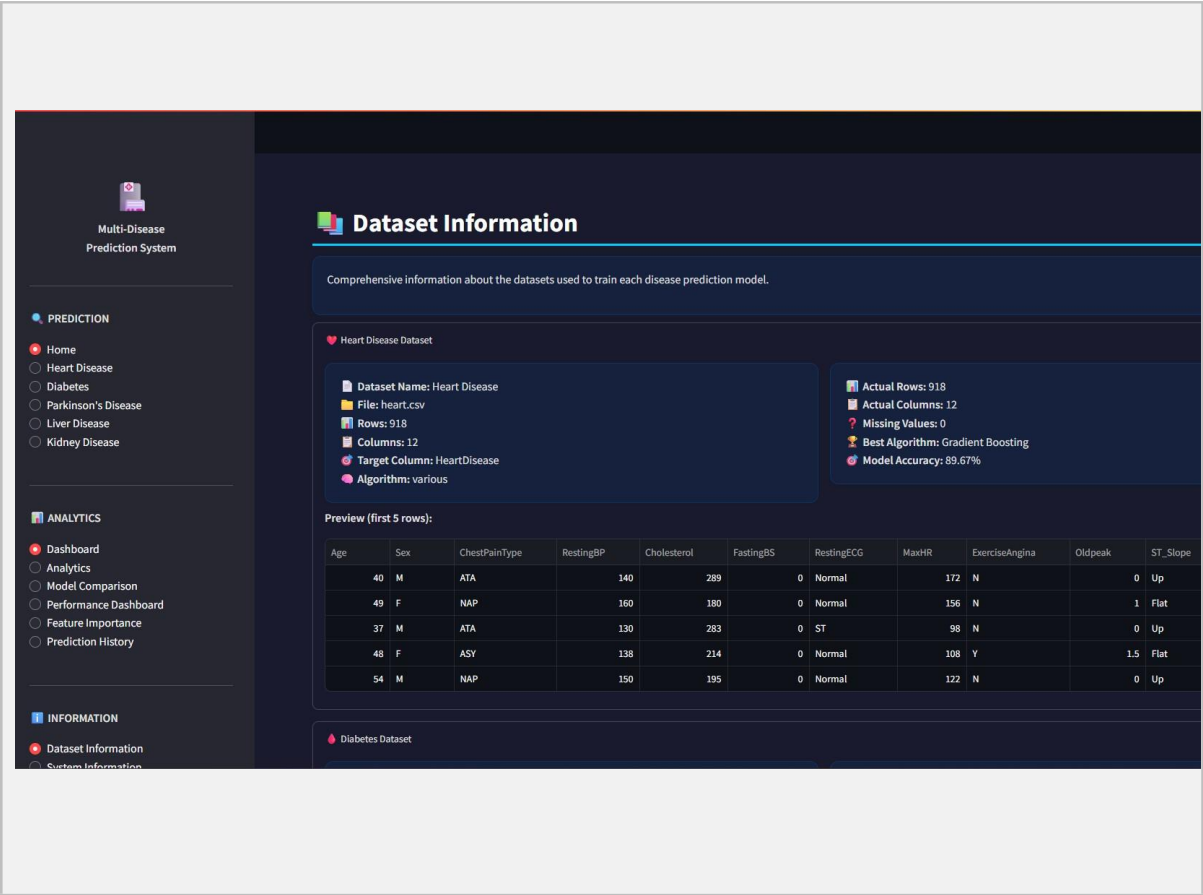
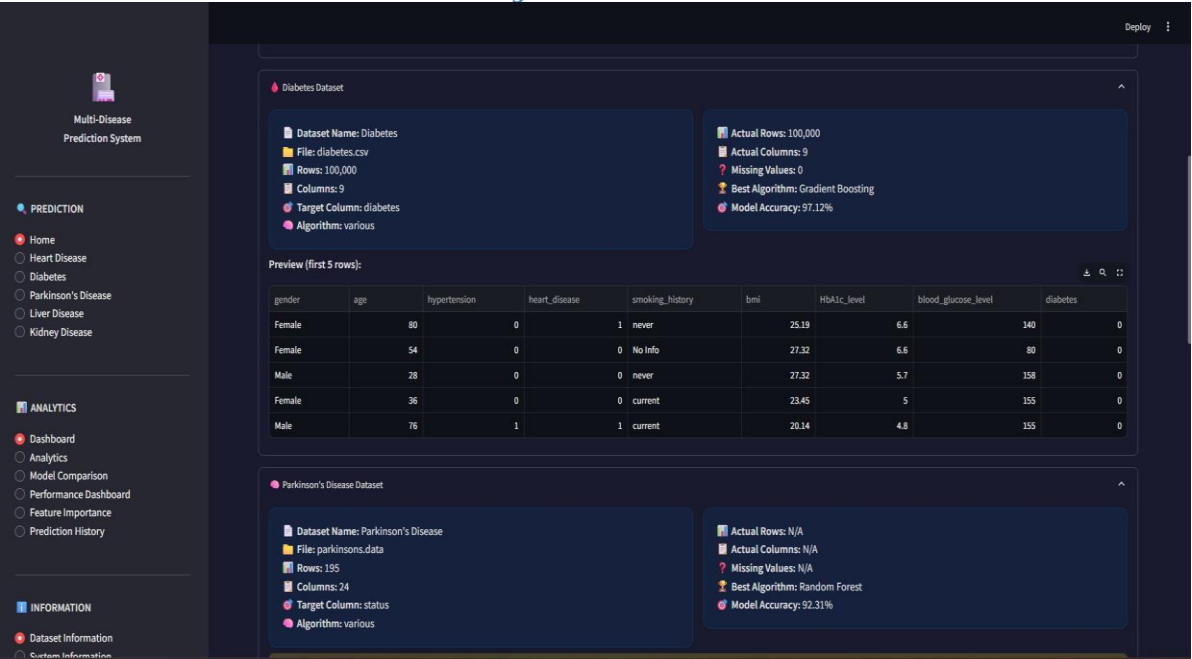


Figure B.1 - Home P



age screenshot placeholder

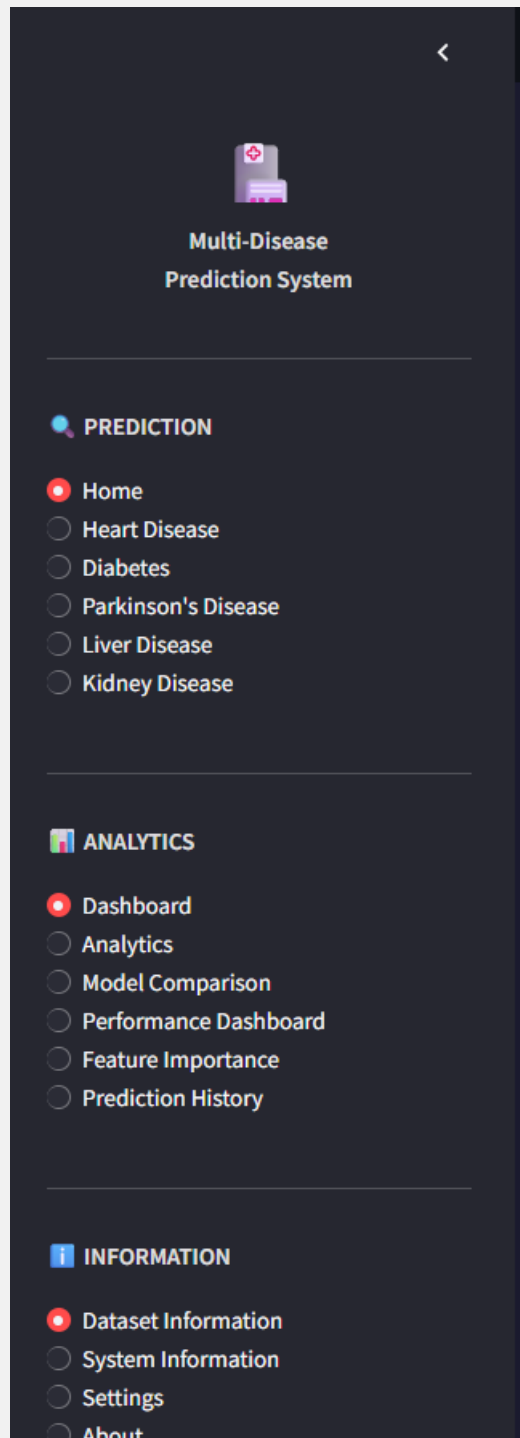


Figure B.2 - Sidebar Navigation screenshot placeholder

Multi-Disease Prediction System

PREDICTION

Home

Heart Disease

Diabetes

Parkinson's Disease

Liver Disease

Kidney Disease

ANALYTICS

Dashboard

Analytics

Model Comparison

Performance Dashboard

Feature Importance

Prediction History

Heart Disease Prediction

About Heart Disease Prediction

Age

30

-

+

Fasting Blood Sugar > 120 mg/dL

No (0)

Exercise Induced Angina

No

Sex

Male

Resting ECG

Normal

ST Depression (Oldpeak)

0.0

Chest Pain Type

ATA (Atypical Angina)

Max Heart Rate Achieved

150

ST Slope

Up

Resting Blood Pressure (mm Hg)

120

Cholesterol (mg/dL)

200

Patient Name (for records)

Unknown

Predict Heart Disease

Reset

Feature Importance Analysis

Figure B.3 - Heart Disease Prediction screenshot placeholder



Multi-Disease
Prediction System

PREDICTION

- Home
- Heart Disease
- Diabetes
- Parkinson's Disease
- Liver Disease
- Kidney Disease

ANALYTICS

- Dashboard
- Analytics
- Model Comparison
- Performance Dashboard
- Feature Importance
- Prediction History

Diabetes Prediction

[About Diabetes Prediction](#)

Gender	Smoking History	HbA1c Level
Male	never	5.5
Age	BMI	Blood Glucose Level (mg/dL)
30	25.00	100
Hypertension		
No		
Heart Disease		
No		
Patient Name (for records)		
Unknown		
Predict Diabetes		Reset
Feature Importance Analysis		

Figure B.4 - Diabetes Prediction screenshot placeholder

Multi-Disease Prediction System

PREDICTION

- Home
- Heart Disease
- Diabetes
- Parkinson's Disease
- Liver Disease
- Kidney Disease

ANALYTICS

- Dashboard
- Analytics
- Model Comparison
- Performance Dashboard
- Feature Importance
- Prediction History

INFORMATION

- Dataset Information
- System Information
- Settings
- About

Quick Stats

Total Predictions: 18

Today: 6

Light Mode

Parkinson's Disease Prediction

About Parkinson's Prediction

MDVP-F0(Hz) - Avg Vocal Freq	MDVP-F0(Hz) - Max Vocal Freq	MDVP-F0(Hz) - Min Vocal Freq	MDVP-Jitter(Hz)	MDVP-Jitter(Sb)	MDVP-RAP	MDVP-RPQ	Shimmer-APQ3	Shimmer-APQ5	MDVP-AQ1	Shimmer-DQA	HNR	HNR	SPQ6	DFA	Spread1	Spread2	D2	PQ6
120.000	150.000	100.000	0.0000	0.0005	0.00300	0.00300	0.01500	0.02000	0.02000	0.04000	0.01000	20.000	0.500	0.800	-4.500	0.300	2.300	0.300

Patient Name (for records)

Unknown

Predict Parkinson's Disease

Reset

Feature Importance Analysis

Figure B.5 - Parkinson's Prediction screenshot placeholder

Click to go forward, hold to see history

Multi-Disease Prediction System

PREDICTION

- Home
- Heart Disease
- Diabetes
- Parkinson's Disease
- Liver Disease
- Kidney Disease

ANALYTICS

- Dashboard
- Analytics
- Model Comparison
- Performance Dashboard
- Feature Importance

Liver Disease Prediction

About Liver Disease Prediction

Age	Alkaline Phosphatase (IU/L)	AST (IU/L)
45	200	30
Gender	ALT (IU/L)	Total Proteins (g/dL)
Male	30	6.5
Total Bilirubin (mg/dL)	Albumin (g/dL)	
1.0	3.5	
Direct Bilirubin (mg/dL)	A/G Ratio	
0.3	1.0	

Patient Name (for records)

Unknown

Predict Liver Disease

Reset

Feature Importance Analysis

Multi-Disease Prediction System using Machine Learning | 72

Figure B.6 - Liver Disease Prediction screenshot placeholder

Multi-Disease Prediction System

PREDICTION

Home

Heart Disease

Diabetes

Parkinson's Disease

Liver Disease

Kidney Disease

ANALYTICS

Dashboard

Analytics

Model Comparison

Performance Dashboard

Feature Importance

Prediction History

INFORMATION

Dataset Information

System Information

Settings

About

Quick Stats

Total Predictions: 18

Todays: 8

Light Mode

Kidney Disease Prediction

About Kidney Disease Prediction

Age	50	Pus Cell Clumps	Not Present	Potassium (mEq/L)	4.0
Blood Pressure (mm Hg)	80	Bacteria	Not Present	Hemoglobin (g/dL)	13.0
Specific Gravity (1.005-1.035)	1.020	Blood Glucose Random (mg/dL)	120	Packed Cell Volume	40
Albumin (g/dl)	0	Blood Urea (mg/dL)	40	White Blood Cell Count	8000
Sugar (g/dl)	0	Serum Creatinine (mg/dL)	1.0	Red Blood Cell Count	4.5
Red Blood Cells	Normal	Sodium (mEq/L)	140	Hypertension	No
Pus Cell	Normal			Diabetes Mellitus	No
				Coronary Artery Disease	No
				Appetite	Good
				Pedal Edema	No
				Anemia	No

Patients Name (for records)

Unknown

Predict Kidney Disease

Reset

Feature Importance Analysis

Figure B.7 - Kidney Disease Prediction screenshot placeholder

Multi-Disease Prediction System using Machine Learning | 73

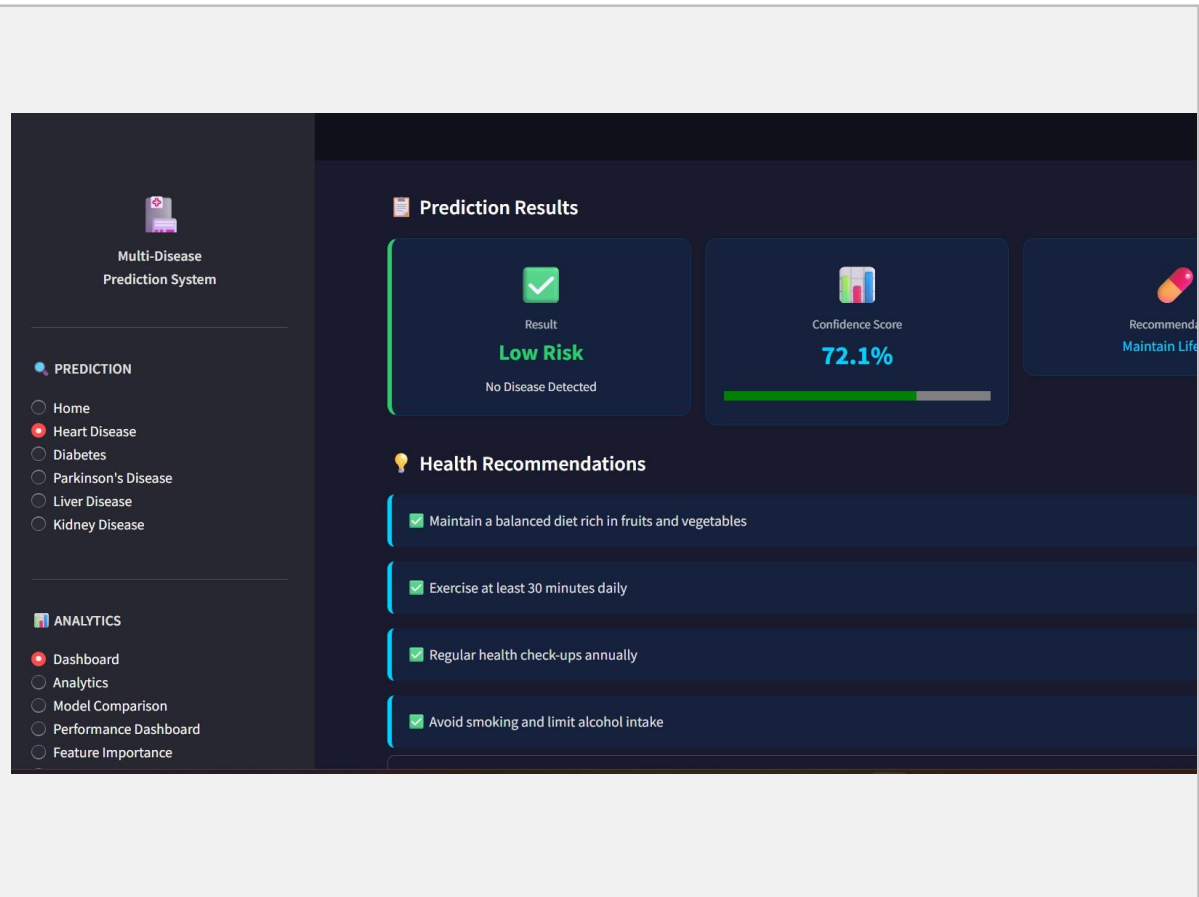


Figure B.8 - Prediction Result screenshot placeholder

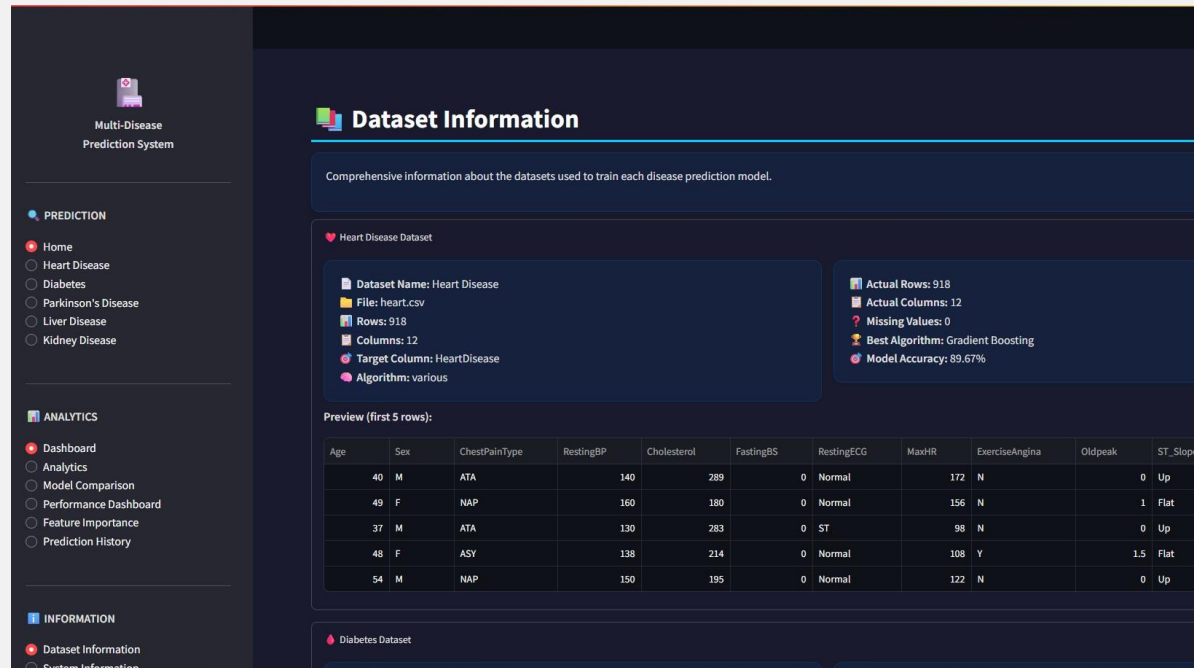


Figure B.9 - Dashboard screenshot placeholder

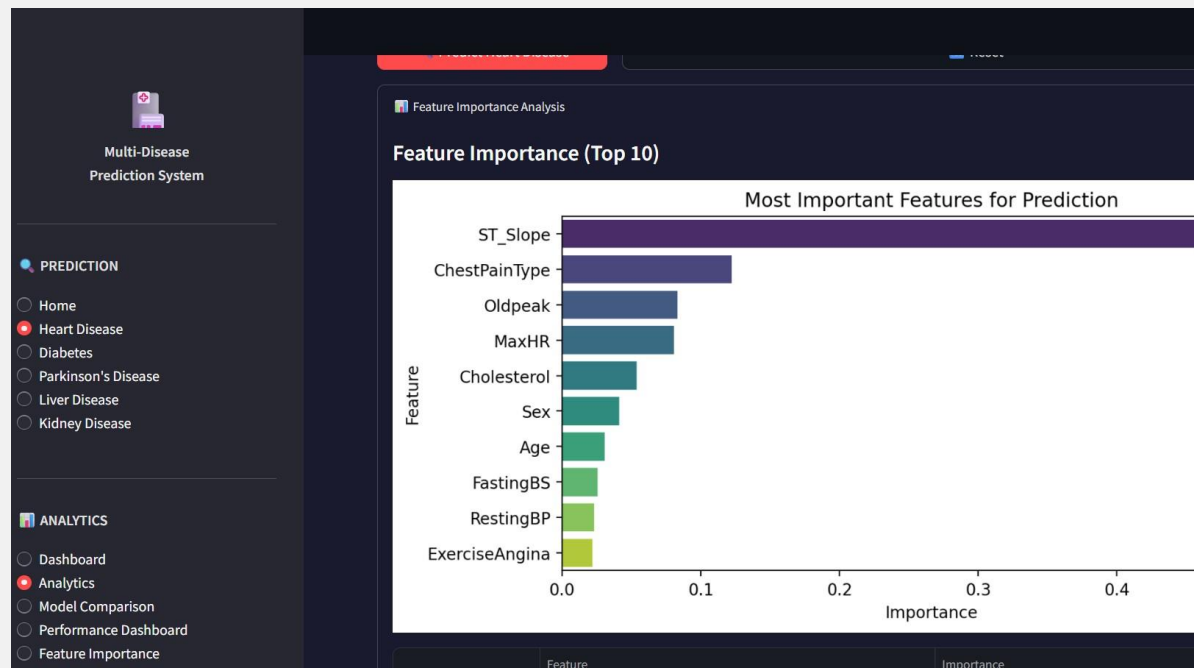


Figure B.10 - Analytics screenshot placeholder

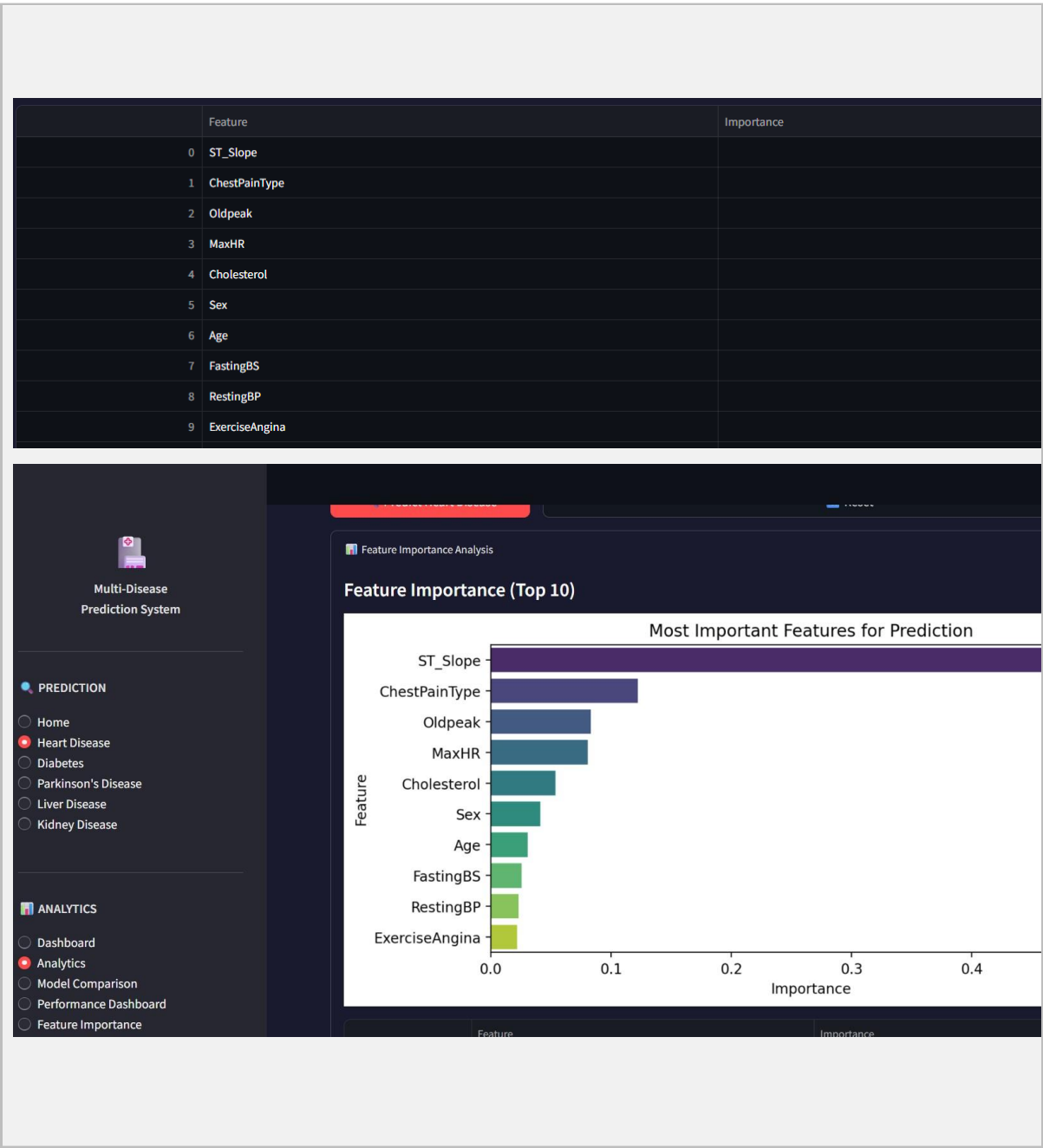
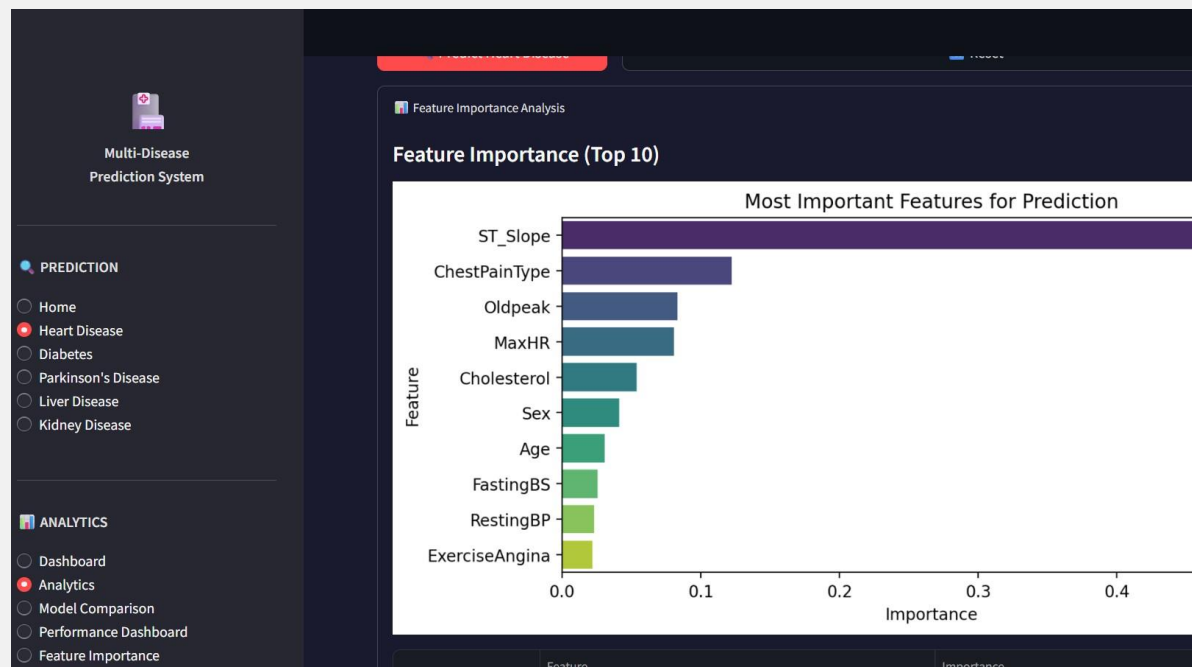


Figure B.11 - Model Comparison screenshot placeholder

[INSERT SCREENSHOT - PERFORMA



NCE DASHBOARD PAGE]

Figure B.12 - Performance Dashboard screenshot placeholder

Figure B.13 - Feature Importance screenshot placeholder

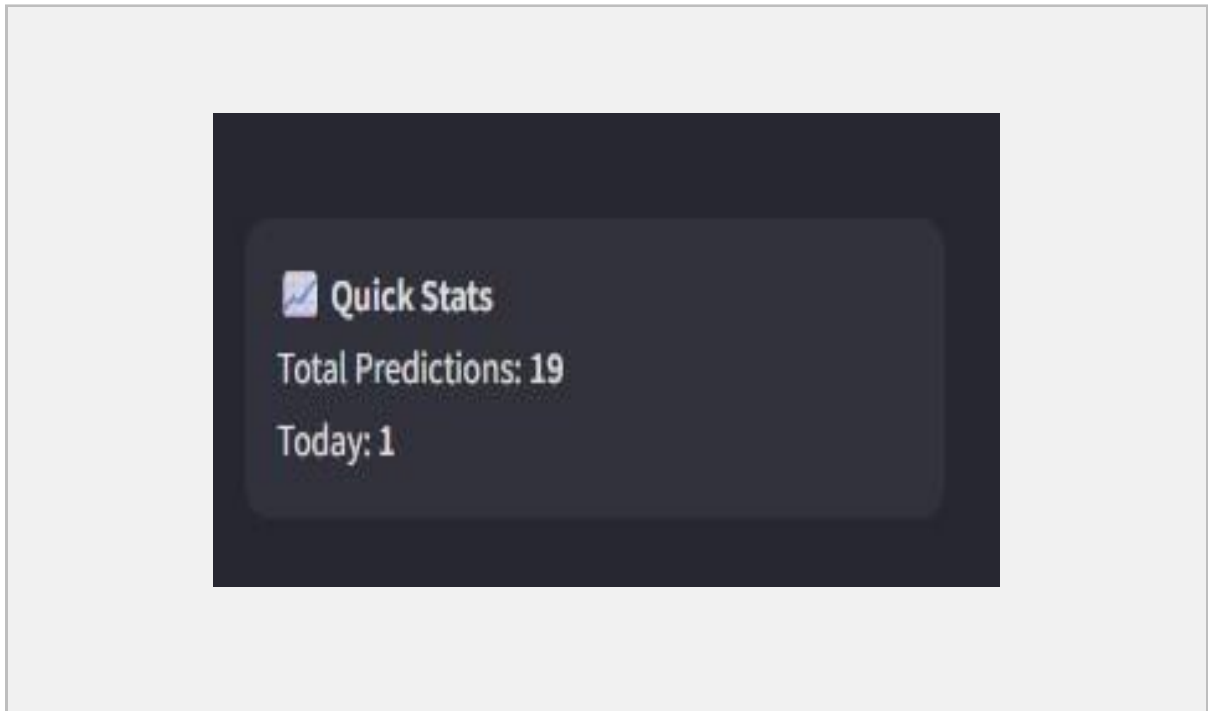


Figure B.14 - Prediction History screenshot placeholder

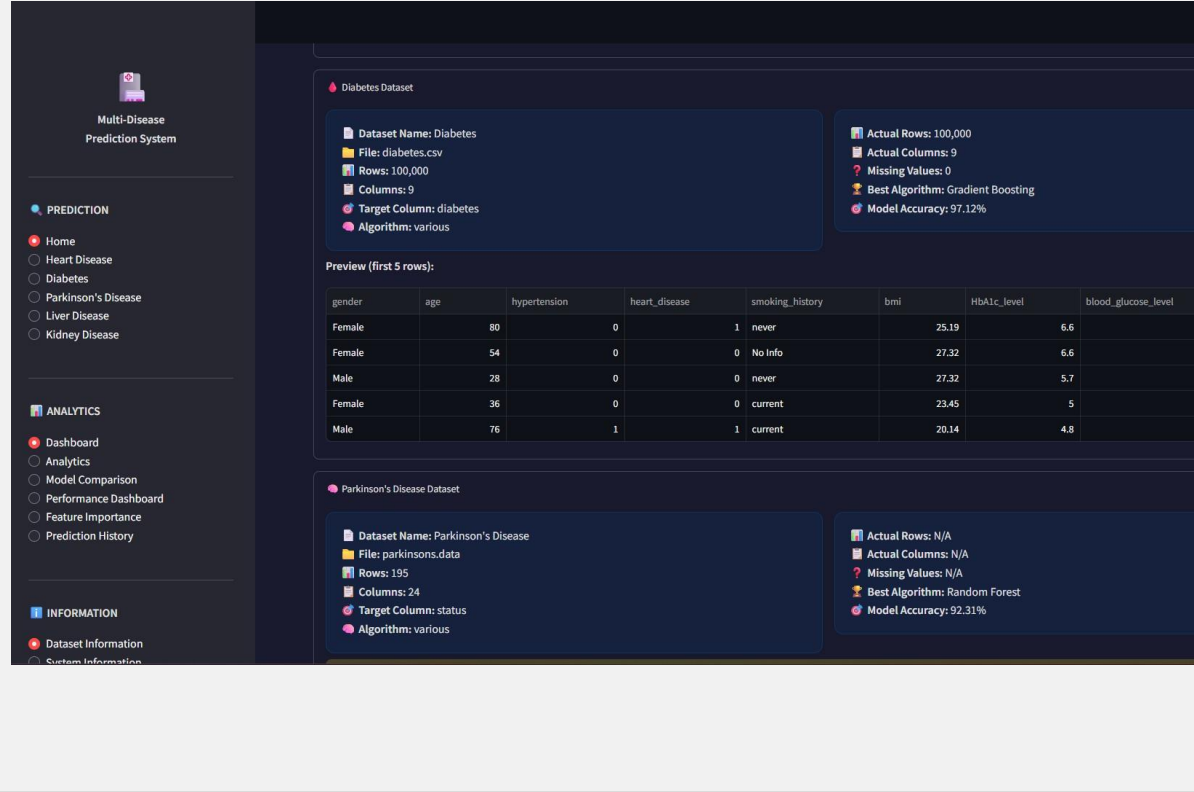


Figure B.15 - Dataset Information screenshot placeholder

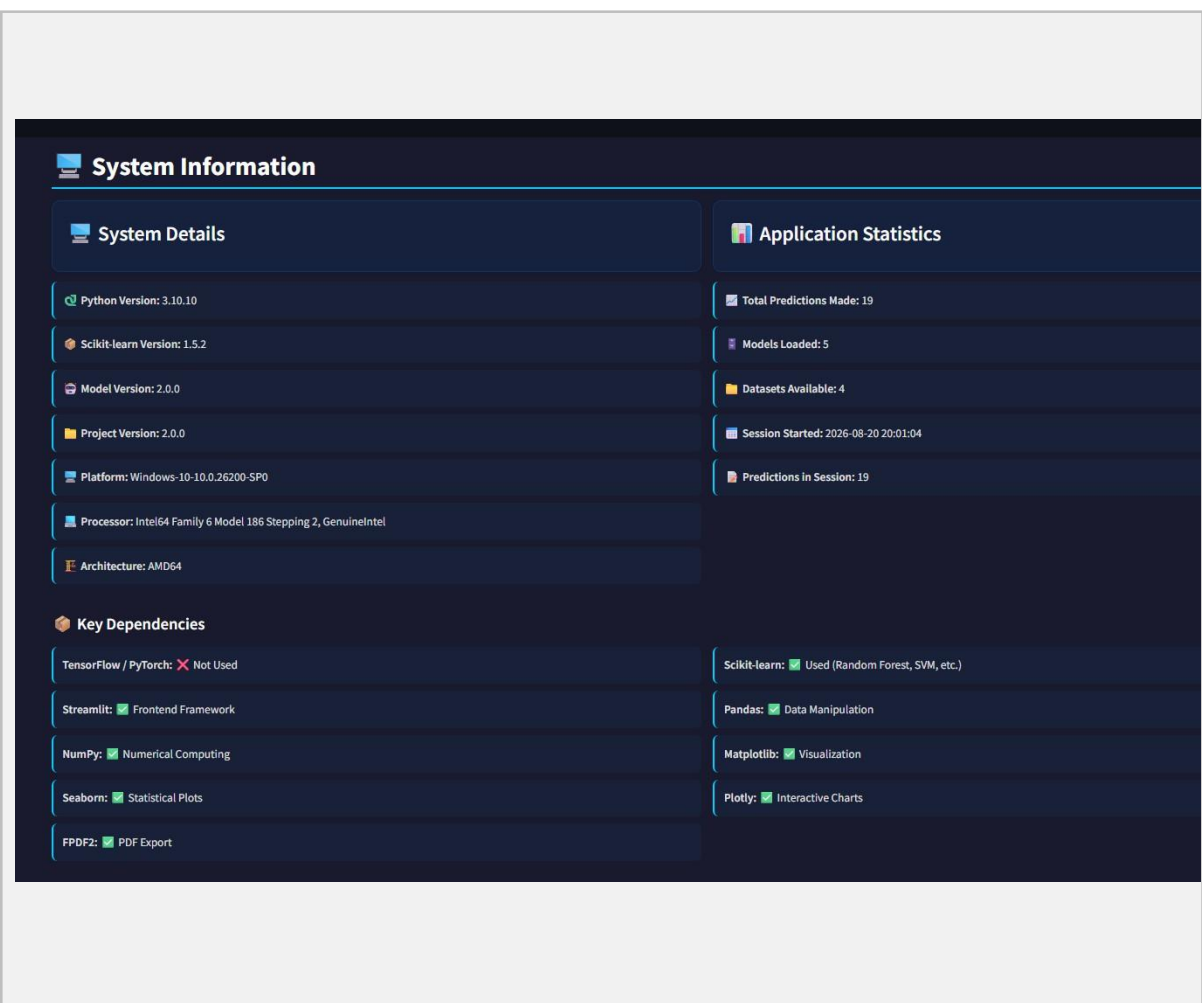


Figure B.16 - System Information screenshot placeholder

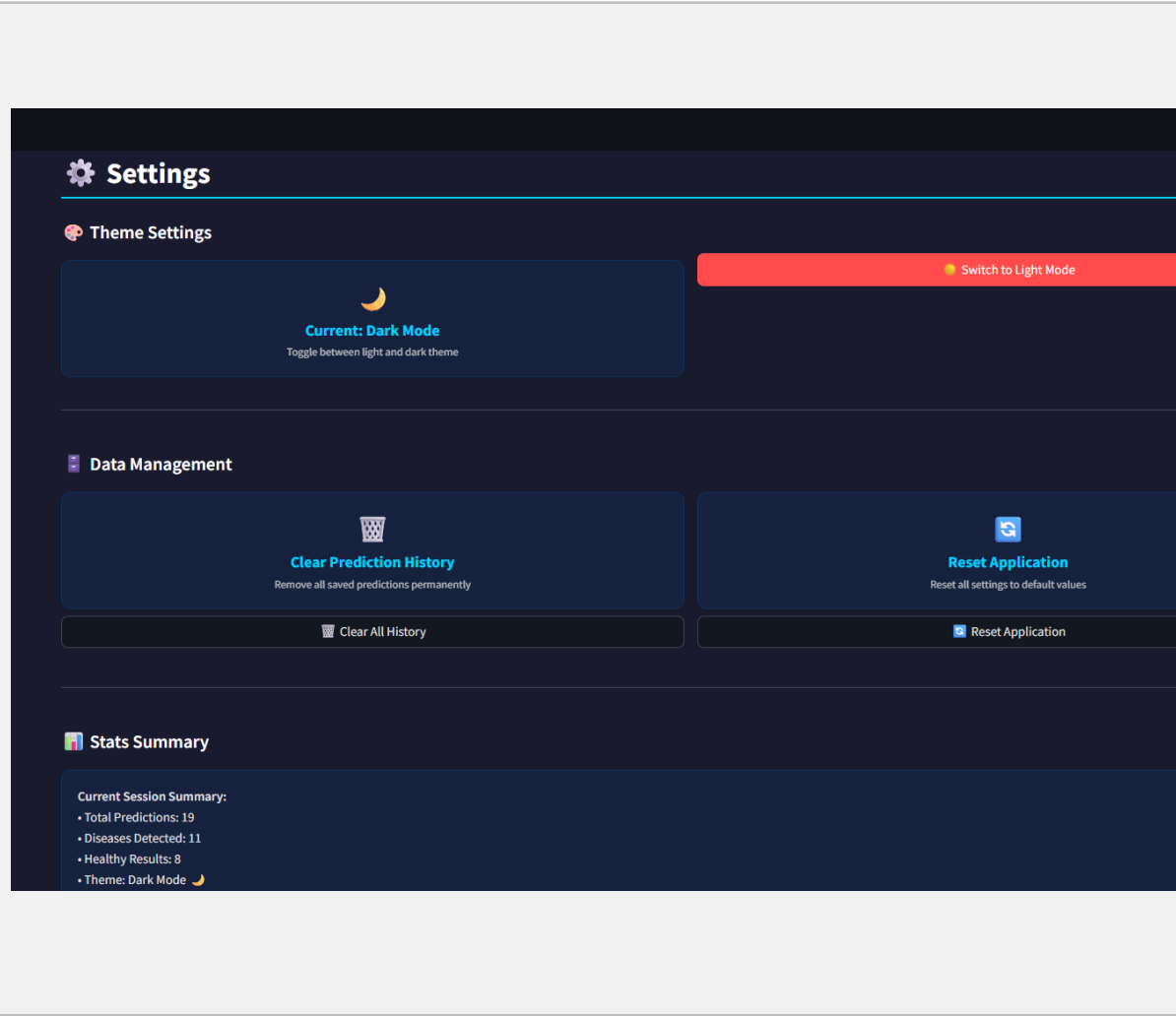


Figure B.17 - Settings screenshot placeholder

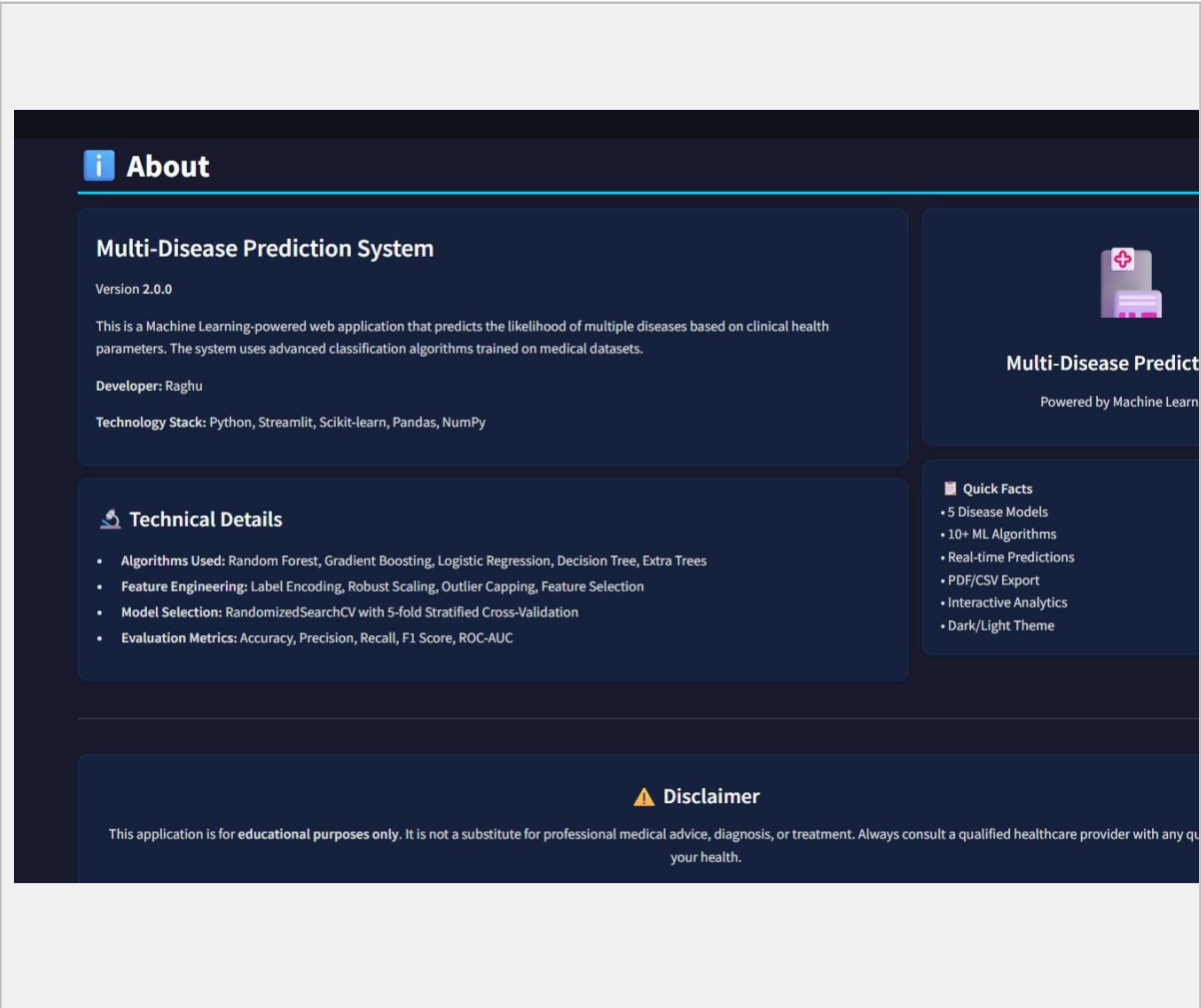


Figure B.18 - About Page screenshot placeholder

APPENDIX C - SAMPLE PREDICTION FLOW

Prediction Workflow

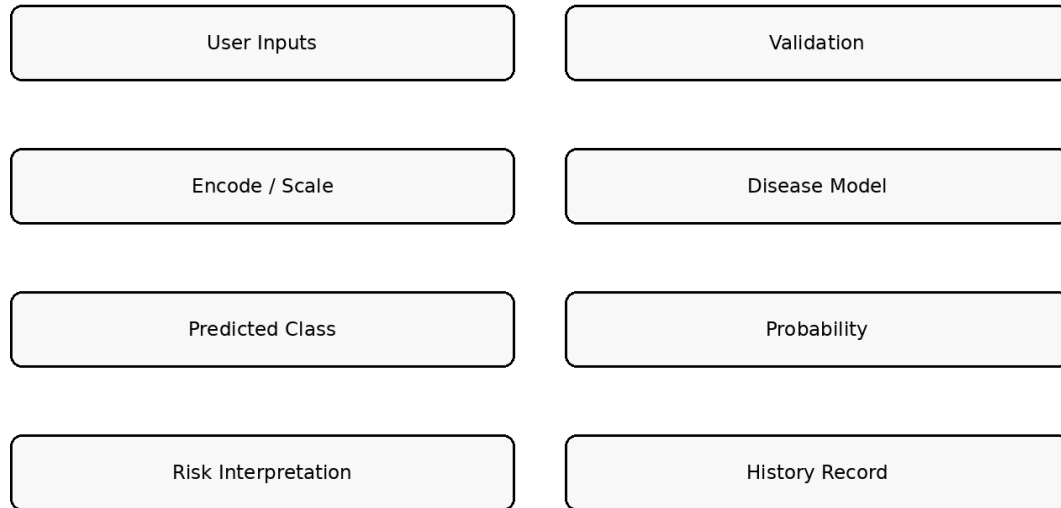


Figure C.1 - Sample Prediction Flow

A typical prediction session begins with disease selection, followed by user input collection and validation. The application then applies the stored preprocessing transformations, invokes the disease-specific model, displays the output, records the prediction in history, and makes the record available to analytics/export features.

APPENDIX D - TESTING

Test Area	Relevant File(s)	Purpose
Model loading	verify_models.py	Confirm model artifacts can be discovered and loaded
Prediction checks	test_models.py	Exercise model inference behavior and consistency
Fix / regression checks	test_fix.py	Validate targeted corrections or regression scenarios
Input validation	Application-level checks	Ensure invalid or incomplete inputs are handled safely
Artifact verification	verify_models.py	Verify required model/scaler/encoder/feature artifacts

- Model loading validation
- Prediction consistency
- Input validation
- Model artifact verification
- Application-level testing
- Basic unit testing
- Error handling

No exact test count or pass rate is claimed because those values were not supplied as verified evidence. The final submission can replace this section with the output of the project test suite.

APPENDIX E - DEPLOYMENT INFORMATION

Item	Value
Organization	TVK Technologies
Domain	AI with Data Science
Internship Period	08 July 2026 - 07 August 2026
Live Application	https://organsenseai.streamlit.app/
GitHub Repository	https://github.com/raghul692/Multi-Prediction-Sytem
Company Website	www.tvktechs.com
Company Email	hr@tvktechs.com
Company Phone	+91 94439 38850
Supervisor	PRIYATHARSHAN A / Program Manager
Completion Letter Date	08 August 2026

Deployment note: the live URL is presented as the supplied project deployment reference. The application remains an educational/research project and must not be represented as a clinical diagnostic service.

--- END OF REPORT ---